

Human review packet: DGD26AdmissionsPredictability

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: DGD26AdmissionsPredictability
Paper id: DGD26AdmissionsPredictability
Source version: AAAI 2026 paper; pinned arXiv TeX source archive
Source record: audit/paper_statement_map.json
Rows: 63
Generated: 2026-08-18
Source URL: [official source](#)

Review status: 0/63 source-claim screens; 0/23 library prerequisite screens. This is a review worksheet while those direct checks remain pending; it is not a final validation receipt.

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) EconCSLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-EconCSLib-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper DGD26AdmissionsPredictability --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's reviewer-owned review record.

Contents

- Semantic library prerequisites (23; not paper claims)
 - [EconCSLib.FiniteChoice.ChoiceRule](#)
 - [EconCSLib.FiniteChoice.Consistent](#)
 - [EconCSLib.FiniteChoice.choiceGainTerm](#)
 - [EconCSLib.FiniteChoice.choiceLossTerm](#)
 - [EconCSLib.FiniteChoice.choiceDistance](#)
 - [EconCSLib.FiniteChoice.DUnstable](#)
 - [EconCSLib.FiniteChoice.Feasible](#)
 - [EconCSLib.FiniteChoice.borderlineSet](#)
 - [EconCSLib.FiniteChoice.waitlistedSet](#)
 - [EconCSLib.FiniteChoice.GeneralVariabilityAtMost](#)
 - [EconCSLib.FiniteChoice.GeneralVariabilityExactly](#)
 - [EconCSLib.FiniteChoice.Independent](#)
 - [EconCSLib.FiniteChoice.Monotonic](#)
 - [EconCSLib.FiniteChoice.QAcceptant](#)
 - [EconCSLib.FiniteChoice.StrictTotalOrder](#)
 - [EconCSLib.FiniteChoice.QRepresentative](#)
 - [EconCSLib.FiniteChoice.Substitutable](#)
 - [EconCSLib.FiniteChoice.TightlyDUnstable](#)
 - [EconCSLib.FiniteChoice.VariabilityAtMost](#)
 - [EconCSLib.FiniteChoice.VariabilityExactly](#)
 - [EconCSLib.FiniteChoice.ZeroUnstable](#)
 - [EconCSLib.FiniteChoice.linearTopQChoice](#)
 - [EconCSLib.FiniteChoice.sequentialComposition](#)
- Source claims (63)
 - [paper_fixed_threshold_formula_statementSpec](#)
 - [paper_choice_function_model_definition_statementSpec](#)
 - [paper_definition_choice_labelSpec](#)

- paper_q_acceptance_definition_statementSpec
- paper_total_order_definition_statementSpec
- paper_definition_choice_distanceSpec
- paper_d_instability_definition_statementSpec
- paper_tight_d_instability_definition_statementSpec
- paper_variability_exactly_definition_statementSpec
- paper_ml_representation_definition_statementSpec
- paper_rank_threshold_formula_statementSpec
- paper_substitutability_definition_statementSpec
- paper_definition_sequential_compositionSpec
- paper_monotonicity_definition_statementSpec
- paper_consistency_definition_statementSpec
- paper_q_representativeness_definition_statementSpec
- paper_zero_instability_definition_statementSpec
- paper_independence_definition_statementSpec
- paper_definition_borderline_setSpec
- paper_definition_waitlisted_setSpec
- paper_general_variability_exactly_definition_statementSpec
- paper_lap_model_definition_statementSpec
- paper_lap_slot_below_definition_statementSpec
- paper_lap_same_slot_order_definition_statementSpec
- paper_lap_assignment_choice_formula_statementSpec
- paper_independent_zero_unstable_statementSpec
- paper_independent_zero_unstable_corollary_statementSpec
- paper_no_consistent_positive_tightly_even_statementSpec
- paper_zero_distance_statementSpec
- paper_calculating_instability_statementSpec
- paper_corrected_consistency_of_removable_sets_statementSpec
- paper_lap_strictly_higher_slot_applicant_assigned_statementSpec
- paper_lap_assigned_strictly_outranks_rejected_statementSpec
- paper_monotonicity_term_statementSpec
- paper_monotonicity_q_acceptance_incompatible_statementSpec
- paper_acceptant_one_instability_variability_borderline_eq_waitlisted_after_changing_insert_statementSpec
- paper_substitutability_term_statementSpec
- paper_non_substitutable_single_add_statementSpec
- paper_ml_fixed_threshold_representation_zero_unstable_statementSpec
- paper_ml_rank_threshold_can_represent_exact_one_statementSpec
- paper_ml_rank_threshold_representation_instability_bound_statementSpec
- paper_ml_rank_threshold_representation_variability_bound_statementSpec
- paper_program_classes_one_instability_statementSpec
- paper_screened_open_variability_one_statementSpec
- paper_screened_open_dia_variability_two_statementSpec
- paper_educational_option_variability_three_statementSpec
- paper_educational_option_dia_variability_six_statementSpec
- paper_sequential_additive_variability_bound_statementSpec
- paper_append_remove_variability_exact_equivalence_statementSpec
- paper_choice_distance_triangle_statementSpec
- paper_even_distance_iff_inconsistent_statementSpec
- paper_independent_substitutable_monotonic_statementSpec
- paper_no_zero_instability_under_capacity_statementSpec
- paper_substitutability_one_instability_equivalence_statementSpec
- paper_theorem1_tight_all_d_statementSpec
- paper_lap_assignment_one_instability_statementSpec
- paper_lap_assignment_slot_order_class_variability_of_unique_global_optima_statementSpec
- paper_q_representative_forward_exact_statementSpec
- paper_q_representative_converse_exact_statementSpec
- paper_sequential_composition_substitutable_statementSpec
- paper_q_acceptant_substitutable_consistent_statementSpec
- paper_sequential_q_representative_variability_range_statementSpec
- paper_variability_one_iff_single_order_statementSpec

Material library prerequisites

EconCSLib.FiniteChoice.ChoiceRule

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$(\alpha : \text{Type } u_2) \rightarrow [\text{DecidableEq } \alpha] \rightarrow \text{Finset } \alpha \rightarrow \text{Finset } \alpha$

Exact Lean library declaration

abbrev ChoiceRule ($\alpha : \text{Type}^*$) [$\text{DecidableEq } \alpha$] := $\text{Finset } \alpha \rightarrow \text{Finset } \alpha$

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.Consistent

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$\forall \{\alpha : \text{Type } u_1\} [\text{inst} : \text{DecidableEq } \alpha] (C : \text{EconCSLib.FiniteChoice.ChoiceRule } \alpha) \{X_1 X_2 : \text{Finset } \alpha\},$
 $C X_2 \subseteq X_1 \rightarrow \neg X_1 \subseteq X_2 \rightarrow C X_2 = C X_1$

Exact Lean library declaration

```
def Consistent (C : ChoiceRule α) : Prop :=  
  ∀ {X₁ X₂}, C X₂ ⊆ X₁ → X₁ ⊆ X₂ → C X₂ = C X₁
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.choiceGainTerm

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{α : Type u_1} →  
  [inst : DecidableEq α] → (C : EconCSLib.FiniteChoice.ChoiceRule α) → (X1 X2 : Finset α) → ((X1 n C X2) \ C X1).card
```

Exact Lean library declaration

```
def choiceGainTerm (C : ChoiceRule α) (X1 X2 : Finset α) : ℕ :=  
  ((X1 n C X2) \ C X1).card
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.choiceLossTerm

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{α : Type u_1} →  
  [inst : DecidableEq α] → (C : EconCSLib.FiniteChoice.ChoiceRule α) → (X1 X2 : Finset α) → (C X1 \ C X2).card
```

Exact Lean library declaration

```
def choiceLossTerm (C : ChoiceRule α) (X1 X2 : Finset α) : ℕ :=  
  (C X1 \ C X2).card
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.choiceDistance

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{α : Type u_1} →  
[inst : DecidableEq α] →  
(C : EconCSLib.FiniteChoice.ChoiceRule α) →  
(X1 X2 : Finset α) → EconCSLib.FiniteChoice.choiceGainTerm C X1 X2 + EconCSLib.FiniteChoice.choiceLossTerm C X1 X2
```

Exact Lean library declaration

```
def choiceDistance (C : ChoiceRule α) (X1 X2 : Finset α) : ℕ :=  
  choiceGainTerm C X1 X2 + choiceLossTerm C X1 X2
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.DUnstable

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {α : Type u_1} [inst : DecidableEq α] (d : ℕ) (C : EconCSLib.FiniteChoice.ChoiceRule α) (X : Finset α),  
  ∀ x ∉ X, EconCSLib.FiniteChoice.choiceDistance C X (insert x X) ≤ d
```

Exact Lean library declaration

```
def DUnstable (d : ℕ) (C : ChoiceRule α) : Prop :=  
  ∀ X x, x ∉ X → choiceDistance C X (insert x X) ≤ d
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.Feasible

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$\forall \{\alpha : \text{Type } u_1\} [\text{inst} : \text{DecidableEq } \alpha] (C : \text{EconCSLib.FiniteChoice.ChoiceRule } \alpha) (X : \text{Finset } \alpha), C \ X \subseteq X$

Exact Lean library declaration

```
def Feasible (C : ChoiceRule  $\alpha$ ) : Prop :=  
   $\forall X, C \ X \subseteq X$ 
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.borderlineSet

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{α : Type u_1} →  
  [inst : DecidableEq α] →  
    [inst_1 : Fintype α] →  
      (C : EconCSLib.FiniteChoice.ChoiceRule α) → (X : Finset α) → Finset.univ.biUnion fun x => C X \ C (insert x X)
```

Exact Lean library declaration

library declaration is not registered or discoverable

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.waitlistedSet

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{α : Type u_1} →  
  [inst : DecidableEq α] →  
  [Fintype α] → (C : EconCSLib.FiniteChoice.ChoiceRule α) → (X : Finset α) → X.biUnion fun x => C (X.erase x) \ C X
```

Exact Lean library declaration

```
def waitlistedSet [Fintype α] (C : ChoiceRule α) (X : Finset α) : Finset α :=  
  X.biUnion fun x => C (X.erase x) \ C X
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.GeneralVariabilityAtMost

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] (m : ℕ) (C : EconCSLib.FiniteChoice.ChoiceRule α)
  (X : Finset α),
  (EconCSLib.FiniteChoice.borderlineSet C X).card ≤ m ∧ (EconCSLib.FiniteChoice.waitlistedSet C X).card ≤ m
```

Exact Lean library declaration

```
def GeneralVariabilityAtMost [Fintype α] (m : ℕ) (C : ChoiceRule α) : Prop :=
  ∀ X, (borderlineSet C X).card ≤ m ∧ (waitlistedSet C X).card ≤ m
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.GeneralVariabilityExactly

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] (m : ℕ) (C : EconCSLib.FiniteChoice.ChoiceRule α),
  EconCSLib.FiniteChoice.GeneralVariabilityAtMost m C ∧
    ((∃ X, (EconCSLib.FiniteChoice.borderlineSet C X).card = m) ∨
     ∃ X, (EconCSLib.FiniteChoice.waitlistedSet C X).card = m)
```

Exact Lean library declaration

```
def GeneralVariabilityExactly [Fintype α] (m : ℕ) (C : ChoiceRule α) : Prop :=
  GeneralVariabilityAtMost m C ∧
    ((∃ X, (borderlineSet C X).card = m) ∨
     (∃ X, (waitlistedSet C X).card = m))
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.Independent

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : EconCSLib.FiniteChoice.ChoiceRule α) (x : α),  
  (∀ (X : Finset α), x ∈ X → x ∈ C X) ∨ ∀ (X : Finset α), x ∈ X → x ∉ C X
```

Exact Lean library declaration

```
def Independent (C : ChoiceRule α) : Prop :=  
  ∀ x, (∀ X, x ∈ X → x ∈ C X) ∨ (∀ X, x ∈ X → x ∉ C X)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.Monotonic

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$\forall \{\alpha : \text{Type } u_1\} [\text{inst} : \text{DecidableEq } \alpha] (C : \text{EconCSLib.FiniteChoice.ChoiceRule } \alpha) \{X_1 X_2 : \text{Finset } \alpha\},$
 $X_1 \subseteq X_2 \rightarrow C X_1 \subseteq C X_2$

Exact Lean library declaration

```
def Monotonic (C : ChoiceRule α) : Prop :=  
  ∀ {X₁ X₂}, X₁ ⊆ X₂ → C X₁ ⊆ C X₂
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.QAcceptant

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$\forall \{\alpha : \text{Type } u_1\} [\text{inst} : \text{DecidableEq } \alpha] (q : \mathbb{N}) (C : \text{EconCSLib.FiniteChoice.ChoiceRule } \alpha) (X : \text{Finset } \alpha),$
 $(C X).card = \min q X.card$

Exact Lean library declaration

```
def QAcceptant (q : ℕ) (C : ChoiceRule α) : Prop :=  
  ∀ X, (C X).card = min q X.card
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.StrictTotalOrder

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {α : Type u_1} (r : α → α → Prop),  
  (∀ (x : α), ¬r x x) ∧ (∀ {x y z : α}, r x y → r y z → r x z) ∧ ∀ {x y : α}, x ≠ y → r x y ∨ r y x
```

Exact Lean library declaration

```
def StrictTotalOrder (r : α → α → Prop) : Prop :=  
  (∀ x, ¬ r x x) ∧  
  (∀ {x y z}, r x y → r y z → r x z) ∧  
  (∀ {x y}, x ≠ y → r x y ∨ r y x)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.QRepresentative

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {α : Type u_1} [inst : DecidableEq α] (q : ℕ) (C : EconCSLib.FiniteChoice.ChoiceRule α),
  ∃ r,
    EconCSLib.FiniteChoice.StrictTotalOrder r ∧
    EconCSLib.FiniteChoice.QAcceptant q C ∧ ∀ {X : Finset α} {x y : α}, x ∈ C X → y ∈ X → y ∉ C X → r x y
```

Exact Lean library declaration

```
def QRepresentative (q : ℕ) (C : ChoiceRule α) : Prop :=
  ∃ r : α → α → Prop,
    StrictTotalOrder r ∧ QAcceptant q C ∧
    ∀ {X x y}, x ∈ C X → y ∈ X → y ∉ C X → r x y
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.Substitutable

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

$\forall \{\alpha : \text{Type } u_1\} [\text{inst} : \text{DecidableEq } \alpha] (C : \text{EconCSLib.FiniteChoice.ChoiceRule } \alpha) \{X_1 X_2 : \text{Finset } \alpha\},$
 $X_1 \subseteq X_2 \rightarrow X_1 \cap C X_2 \subseteq C X_1$

Exact Lean library declaration

`def Substitutable (C : ChoiceRule α) : Prop :=`
 `∀ {X₁ X₂}, X₁ ⊆ X₂ → X₁ ∩ C X₂ ⊆ C X₁`

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.TightlyDUnstable

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {α : Type u_1} [inst : DecidableEq α] (d : ℕ) (C : EconCSLib.FiniteChoice.ChoiceRule α),  
  EconCSLib.FiniteChoice.DUnstable d C ∧ ∀ k < d, ¬EconCSLib.FiniteChoice.DUnstable k C
```

Exact Lean library declaration

```
def TightlyDUnstable (d : ℕ) (C : ChoiceRule α) : Prop :=  
  DUnstable d C ∧ ∀ k, k < d → ¬ DUnstable k C
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.VariabilityAtMost

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] (m : ℕ) (C : EconCSLib.FiniteChoice.ChoiceRule α)
  (X : Finset α), (EconCSLib.FiniteChoice.borderlineSet C X).card ≤ m
```

Exact Lean library declaration

```
def VariabilityAtMost [Fintype α] (m : ℕ) (C : ChoiceRule α) : Prop :=
  ∀ X, (borderlineSet C X).card ≤ m
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.VariabilityExactly

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] (m : ℕ) (C : EconCSLib.FiniteChoice.ChoiceRule α),  
  EconCSLib.FiniteChoice.VariabilityAtMost m C ∧ ∃ X, (EconCSLib.FiniteChoice.borderlineSet C X).card = m
```

Exact Lean library declaration

```
def VariabilityExactly [Fintype α] (m : ℕ) (C : ChoiceRule α) : Prop :=  
  VariabilityAtMost m C ∧ ∃ X, (borderlineSet C X).card = m
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.ZeroUnstable

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : EconCSLib.FiniteChoice.ChoiceRule α) {X₁ X₂ : Finset α},  
  X₁ ⊆ X₂ → EconCSLib.FiniteChoice.choiceDistance C X₁ X₂ = 0
```

Exact Lean library declaration

```
def ZeroUnstable (C : ChoiceRule α) : Prop :=  
  ∀ {X₁ X₂}, X₁ ⊆ X₂ → choiceDistance C X₁ X₂ = 0
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.linearTopQChoice

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{α : Type u_1} →  
  [inst : DecidableEq α] →  
    [inst_1 : LinearOrder α] →  
      (q : ℕ) →  
        (a : Finset α) →  
          if h : q ≤ a.card then Finset.image (fun i => ↑((a.orderIso0ffFin ↪) {↑i, ...})) Finset.univ else a
```

Exact Lean library declaration

```
noncomputable def linearTopQChoice (q : ℕ) : ChoiceRule α :=  
  fun X =>  
    if h : q ≤ X.card then  
      Finset.univ.image  
        (fun i : Fin q =>  
          ((X.orderIso0ffFin (by rfl)  
            (i.1, lt_of_lt_of_le i.2 h) : X) : α))  
    else  
      X
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

EconCSLib.FiniteChoice.sequentialComposition

Source connection: no explicit byte-pinned paper source connection is registered.

Lean-expanded library semantic target

```
{α : Type u_1} →
[inst : DecidableEq α] →
(a : List (EconCSLib.FiniteChoice.ChoiceRule α)) →
(a_1 : Finset α) →
  List.brecOn a
    (fun x f =>
      (match (motive :=
        (x : List (EconCSLib.FiniteChoice.ChoiceRule α)) → List.below x → EconCSLib.FiniteChoice.ChoiceRule α)
        x with
      | [] => fun x x_1 => ∅
      | C :: Cs => fun x X =>
        let chosen := C X;
        chosen u x.1 (X \ chosen))
      f)
    a_1
```

Exact Lean library declaration

```
def sequentialComposition : List (ChoiceRule α) → ChoiceRule α
| [] => fun _ => ∅
| C :: Cs =>
  fun X =>
    let chosen := C X
    chosen u sequentialComposition Cs (X \ chosen)

/-- Borderline admits displaced by some single added applicant. -/
def borderlineSet [Fintype α] (C : ChoiceRule α) (X : Finset α) : Finset α :=
  Finset.univ.biUnion fun x => C X \ C (insert x X)
```

Recorded source-to-library screening Verdict: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 162-170

Verbatim source input

Training a machine learning model means selecting a f from a set of possible models, i.e., a hypothesis class. Generally, this is done by finding a model which achieves low error on available data (a train set). It is typical to use continuous optimization algorithms to first train a continuous score function $s : \mathcal{X} \rightarrow [0,1]$ by minimizing the error. Then the classification model is defined as

$$\begin{aligned} &f(x) = \mathbf{1}\{s(x) \geq t\} \end{aligned}$$

based on a fixed threshold value t , often $t=0.5$. Notice that models of the form ml-independent predict outcomes independently for each applicant. At test time inference, they do not consider the overall applicant pool -- they implicitly model the training time pool, through its effect on the training labels.

Expanded PaperInterface specification

$\forall \{\alpha : \text{Type } u_1\} [\text{inst} : \text{DecidableEq } \alpha] (\text{score} : \alpha \rightarrow \mathbb{R}) (\text{threshold} : \mathbb{R}) (X : \text{Finset } \alpha) (x : \alpha),$
 $x \in \text{DGD26AdmissionsPredictability.paperFixedThresholdChoice score threshold } X \leftrightarrow$
 $x \in X \wedge \text{DGD26AdmissionsPredictability.paper_definition_fixed_threshold_predictor score threshold } X \ x$

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_fixed_threshold_formula_statement

Recorded source-to-Spec screening **Verdict:** not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 172-181

Verbatim source input

`\subsection{Choice Functions}`

Many admissions processes have capacity constraints (either perceived or actual). This fact gives rise to what we call

→ `\textit{cohort-dependence}`: an applicant's acceptance decision depends on not only their own characteristics, but also the

→ number and characteristics of other applicants.

`\textit{Choice functions}` formalize admission decision processes and allow modeling such dependence.

A choice function describes the behavior of ``choosing'' elements from a set of available options, and is commonly used in

→ economic models. Formally, a choice function over the universe $\mathcal{X}$ is a function $\textit{choice}: 2^{\mathcal{X}} \rightarrow 2^{\mathcal{X}}$ that maps

→ all subsets X of $\mathcal{X}$ to the accepted set $\textit{choice}(X)$, where $\textit{choice}(X) \subseteq X$. As our motivation focuses on

→ real applicant datasets, we consider finite subsets.

Recalling admissions data, the binary admissions label y for student x is 1 if $x \in \textit{choice}(X)$ and 0 otherwise.

To aid in our discussion, we will slightly abuse notation to relate pairs (x_i, y_i) to the choice function $\textit{choice}(X)$

→ where $x_i \in X$.

Define $\textit{choice}(X)_i$ as 1 if $x_i \in \textit{choice}(X)$ and 0 otherwise; this allows us to write the label $y_i = \textit{choice}(X)_i$.

Expanded PaperInterface specification

$\forall \{ \alpha : \text{Type } u_1 \} [\text{inst} : \text{DecidableEq } \alpha] (C : \text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha),$
 $\text{DGD26AdmissionsPredictability.paper_feasible } C \leftrightarrow \forall (X : \text{Finset } \alpha), C \ X \subseteq X$

Lean proof endpoint (built separately)

`DGD26AdmissionsPredictability.PaperInterface.paper_choice_function_model_definition_statement`

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 179-181

Verbatim source input

Recalling admissions data, the binary admissions label y for student x is 1 if $x \in \text{choice}(X)$ and 0 otherwise. To aid in our discussion, we will slightly abuse notation to relate pairs (x_i, y_i) to the choice function $\text{mathcal{C}}(X) \hookrightarrow$ where $x_i \in X$. Define $\text{mathcal{C}}(X)_i$ as 1 if $x_i \in \text{mathcal{C}}(X)$ and 0 otherwise; this allows us to write the label $y_i = \text{mathcal{C}}(X)_i$.

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) (X : Finset α) (x : α),
  DGD26AdmissionsPredictability.paperChoiceLabel C X x = if x ∈ C X then 1 else 0
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_definition_choice_label

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 186-189

Verbatim source input

```
\begin{restatable}[$q$-Acceptance]{definition}{qacceptance}
\label{def:q-acceptance}
Choice function $\textit{choice}$ is $q$-acceptant if $|\textit{choice}(X)| = \min\{q, |X|\}$ for all input sets $X$.
\end{restatable}
```

[Next verbatim source excerpt]

```
\qacceptance*
\textit{Intuition}: Capacity of $q$.

\substitutability*
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (q : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_q_acceptance q C ↔ ∀ (X : Finset α), (C X).card = min q X.card
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_q_acceptance_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 193-195

Verbatim source input

```
\begin{definition}[Total order] \label{def:total_order}
  A \emph{total order} is a transitive, asymmetric, and complete binary relation  $\text{\succ}$  over elements of  $\text{calX}$ ; that is,
   $\hookrightarrow x_1 \text{\succ} x_2$  implies  $x_2 \not\text{\succ} x_1$ ,  $x_1 \text{\succ} x_2$  and  $x_2 \text{\succ} x_3$  implies  $x_1 \text{\succ} x_3$ , and all  $x_1$ ,
   $\hookrightarrow x_2$  are comparable.
\end{definition}
```

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

A strict total order is asymmetric and transitive, and compares every distinct pair of applicants.

Archival source anchor: cited publication, lines 193-195

Recorded basis DGD source clarification: strict-order completeness ranges over distinct applicant pairs.

Expanded PaperInterface specification

```
∀ {α : Type u_1} (r : α → α → Prop),
  DGD26AdmissionsPredictability.paper_definition_total_order r ↔
  (∀ (x : α), ¬r x x) ∧ (∀ {x y z : α}, r x y → r y z → r x z) ∧ ∀ {x y : α}, x ≠ y → r x y ∨ r y x
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_total_order_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 6

Source locator: cited publication, lines 213-218

Verbatim source input

```
\begin{restatable}[Choice Distance]{definition}{distance} \label{def:distance}
For a choice function  $\text{\choice}$  and sets  $X_1 \text{\subteq} X_2$ , we define the choice distance as:
\begin{equation}
r_{\text{\choice}(X_1, X_2)} := |X_1 \cap \text{\choice}(X_2) \setminus \text{\choice}(X_1)| + |\text{\choice}(X_1) \setminus \text{\choice}(X_2)|
\end{equation}
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) (X₁ X₂ : Finset α),
  EconCSLib.FiniteChoice.choiceDistance C X₁ X₂ = ((X₁ ∩ C X₂) \ C X₁).card + (C X₁ \ C X₂).card
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_definition_choice_distance

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 222-225

Verbatim source input

```
\begin{restatable}[ $d$ -Instability]{definition}{defstability}
\label{def:d-instability}
Choice function  $\text{\textit{choice}}$  is  $d$ -unstable if for all  $X$ , and  $X' = X \cup \{x'\}$ , we have  $r_{\text{\textit{choice}}}(X, X') \leq d$  for any
 $\leftrightarrow$  nonnegative integer  $d$ .  $\text{\textit{choice}}$  is called tightly  $d$ -unstable if it is  $d$ -unstable but not  $(d-1)$ -unstable.
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (d : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_d_instability d C ↔
    ∀ (X : Finset α), ∀ x ∉ X, EconCSLib.FiniteChoice.choiceDistance C X (insert x X) ≤ d
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_d_instability_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: cited publication, lines 222-225

Verbatim source input

```
\begin{restatable}[ $d$ -Instability]{definition}{defstability}
\label{def:d-instability}
Choice function  $\text{\textit{choice}}$  is  $d$ -unstable if for all  $X$ , and  $X' = X \cup \{x'\}$ , we have  $r_{\text{\textit{choice}}}(X, X') \leq d$  for any
 $\leftrightarrow$  nonnegative integer  $d$ .  $\text{\textit{choice}}$  is called tightly  $d$ -unstable if it is  $d$ -unstable but not  $(d-1)$ -unstable.
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (d : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_tight_d_instability d C ↔
  DGD26AdmissionsPredictability.paper_definition_d_instability d C ∧
  ∀ k < d, ¬DGD26AdmissionsPredictability.paper_definition_d_instability k C
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_tight_d_instability_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 235-243

Verbatim source input

We formalize\footnote{For a more general definition of variability (beyond 1-unstable and q -acceptant), see
 $\hookrightarrow$ \Cref{sec:variability-appendix}.} this intuition with the following definition.

```
\begin{restatable}[Variability]{definition}{defvariability}
\label{def:variability}
A  $1$ -unstable,  $q$ -acceptant choice function  $\text{\textit{choice}}$  has variability  $m$  where
\begin{equation}
m \coloneqq \max_{X \subset \text{cal}X} \left| \bigcup \limits_{x' \in \text{cal}X} \text{\textit{choice}}(X) \setminus \text{\textit{choice}}(X \cup \{x'\}) \right|
\end{equation}
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] (m : ℕ)
(C : DGD26AdmissionsPredictability.PaperChoiceRule α),
DGD26AdmissionsPredictability.paper_definition_variability_exactly m C ↔
(∀ (X : Finset α), (DGD26AdmissionsPredictability.paper_definition_borderline_set C X).card ≤ m) ∧
∃ X, (DGD26AdmissionsPredictability.paper_definition_borderline_set C X).card = m
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_variability_exactly_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: cited publication, lines 248-251

Verbatim source input

We first relate the concepts of instability and variability to the practice of machine learning. We focus on the representation capabilities of ML models: when is it possible for an ML model to faithfully represent an admissions process? Formally, we say that an ML model can represent a choice function $\mathcal{C}$ if there exists a function f such that for all applicant sets X , $f(x_i) = \mathcal{C}(X)_i$. {Note that by focusing on representation, our theory is distribution agnostic, and applies to settings where applicant distributions are affected considerations such as strategic behavior and matching algorithms.}

Expanded PaperInterface specification

$\forall \{ \alpha : \text{Type } u_1 \} [\text{inst} : \text{DecidableEq } \alpha] \text{ (predicts : DGD26AdmissionsPredictability.PaperPoolPredictor } \alpha)$
 $(C : \text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha),$
 $\text{DGD26AdmissionsPredictability.paper_definition_ml_representation predicts } C \leftrightarrow$
 $\forall (X : \text{Finset } \alpha), \forall x \in X, \text{ predicts } X \ x \leftrightarrow x \in C\ X$

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_ml_representation_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 11

Source locator: cited publication, lines 253-259

Verbatim source input

Before stating this result, consider a straightforward way to adapt a model~\eqref{eq:ml-independent} to the capacity-constrained setting.
Given that the ML model is based on continuous scores, we can adjust the decisions to account for a known capacity constraint if we have available the set of all test-time applications.
In particular, we can rank the applicants by their scores and then select the top q .
This process results in decisions determined by a cohort dependent threshold $t_q(X)$ which is the score of the q th ranked applicant:
$$f(x; X) = \mathbf{1}\{s(x) \geq t_q(X)\}.$$

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (q : ℕ) (score : α → ℝ) (hinjective : Function.Injective score),
  0 < q →
  ∀ (X : Finset α),
    ∃ threshold,
      ∀ x ∈ X,
        DGD26AdmissionsPredictability.paper_definition_rank_threshold_predictor q score hinjective X x ↔
          threshold ≤ score x
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_rank_threshold_formula_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 12

Source locator: cited publication, lines 273-276

Verbatim source input

```
\begin{restatable}[Substitutability]{definition}{substitutability}
\label{def:substitutability}
Choice function  $\text{\choice}$  is substitutable if  $X_1 \subseteq X_2$  implies  $X_1 \cap \text{\choice}(X_2) \subseteq \text{\choice}(X_1)$ .
\end{restatable}
```

[Next verbatim source excerpt]

```
\substitutability*
\textit{Intuition}: Students selected from a larger pool remain selected when the pool shrinks.
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_substitutability C ↔ ∀ {X₁ X₂ : Finset α}, X₁ ⊆ X₂ → X₁ ∩ C X₂ ⊆ C X₁
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_substitutability_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 13

Source locator: cited publication, lines 307-311

Verbatim source input

$$\begin{array}{l} \text{\textit{A choice function } χ is a \textit{sequential composition} of functions } \chi_1, \dots, \chi_n \text{ if:}} \\ \text{\textit{$\chi(X) = \chi_1(X) \cup \chi_2(\chi_1(X)) \cup \dots \cup \chi_n(\chi_{n-1}(X))$}} \\ \text{\textit{where } $X_{i+1} = X_i \setminus \chi_i(X_i)$ and } X_1 = X$.} \end{array}$$

[Next verbatim source excerpt]

```

\subsubsection{Sequential Composition} \label{sec:seq-proofs}
A useful class of choice functions are choice functions that can be broken down or defined as a sequential composition of
→ functions  $\text{\texttt{\$choice\_n}}, \text{\texttt{\$choice\_n-1}}, \dots, \text{\texttt{\$choice\_1}}$ ; that is,
 $\text{\texttt{\$choice}}(X) = \text{\texttt{\$choice\_n}}(X) \cup \text{\texttt{\$choice\_n-1}}(X_{\text{\texttt{\$n-1}}}) \cup \dots \cup \text{\texttt{\$choice\_1}}(X_1)$  where  $X_i = X_{i+1}$ 
→  $\text{\texttt{\$setminus}}\text{\texttt{\$choice}}\{\text{\texttt{\$i+1}}\}(X_{\text{\texttt{\$i+1}}})$  and  $X_n = X$ . Sequential composition is particularly useful because it preserves a variety
→ of useful properties. Sequences of  $q_i$ -acceptant functions are  $q$ -acceptant, because no duplicate elements are selected by
→ multiple functions, and therefore preserves  $q$ -acceptance when each function in the sequence has capacity  $q_i$  (i.e.,  $q =$ 
 $q_n + \dots + q_1$ .)
\seqcomposition*

```

Expanded PaperInterface specification

```

V {α : Type u_1} [inst : DecidableEq α] (Cs : List (DGD26AdmissionsPredictability.PaperChoiceRule α)),
DGD26AdmissionsPredictability.paper_sequential_composition Cs =
  DGD26AdmissionsPredictability.sequentialCompositionSource Cs

```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper definition sequential composition

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

Review row 14

Source locator: cited publication, lines 484-487

Verbatim source input

```
\begin{restatable}[Monotonicity]{definition}{monotonicity}
\label{def:monotonicity}
Choice function  $\text{\choice}$  is  $\text{\textit{monotonic}}$  if  $X_1 \subseteq X_2$  implies  $\text{\choice}(X_1) \subseteq \text{\choice}(X_2)$ .
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_monotonicity C ↔ ∀ {X₁ X₂ : Finset α}, X₁ ⊆ X₂ → C X₁ ⊆ C X₂
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_monotonicity_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 15

Source locator: cited publication, lines 492-495

Verbatim source input

```
\begin{restatable}[Consistency]{definition}{consistency}
\label{def:consistency}
Choice function  $\text{\choice}$  is \textit{consistent} if  $\text{\choice}(X_2) \subseteq X_1 \subseteq X_2$  implies  $\text{\choice}(X_2) = \text{\choice}(X_1)$ .
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_consistency C ↔ ∀ {X₁ X₂ : Finset α}, C X₂ ⊆ X₁ → X₁ ⊆ X₂ → C X₂ = C X₁
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_consistency_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 16

Source locator: cited publication, lines 500-507

Verbatim source input

```
\begin{restatable}[ $q$ -representativeness]{definition}{qrepresentativeness}
\label{def:q-representative}
 $q$ -acceptant choice function  $\text{\textit{choice}}$  is  $q$ -representative if there exists a strict total ordering  $\text{\textit{succ}}$  over all  $x$  \in
 $\mathcal{X}$  and  $\text{\textit{choice}}(X)$  is the top  $q$  elements of  $X$  for all  $X \subseteq \mathcal{X}$ .
\end{restatable}

\textit{Intuition}: Students are selected according to an exact ranking.

\textbf{Note:} We refer to this property in the main body as being ``characterized by a total order'', or a ``queue'', but this
 $\hookrightarrow$  language --- and definition as a property in its own right --- is seen in, e.g., \cite{echenique2015control}.

[Next verbatim source excerpt]

Total orders are frequently used in modeling choice mechanisms; they are often called \textit{preference orderings} or
 $\hookrightarrow$  \textit{priority orders} in economic models.
Intuitively, total orders induce a ranking over candidates and could result from, e.g., ordering applicants based on their GPA.
We will say that a  $q$ -acceptant choice function  $\text{\textit{C}}$  is \textit{characterized by a total order} if there exists a total
 $\hookrightarrow$  order  $\text{\textit{succ}}$  such that for any input  $X$ , the accepted applicants are the top- $q$  elements of  $X$  ordered by  $\text{\textit{succ}}$ .
In other words,  $x_1 \text{\textit{succ}} x_2$  for all  $x_1 \in \text{\textit{choice}}(X)$ ,  $x_2 \notin \text{\textit{choice}}(X)$ .
We will refer to the process of sorting elements according to a total order  $\text{\textit{succ}}$  and selecting the top  $q$  as a \textit{queue}.
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (q : ℕ) (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_q_representativeness q C  $\leftrightarrow$ 
  ∃ r,
    DGD26AdmissionsPredictability.paper_definition_total_order r  $\wedge$ 
    DGD26AdmissionsPredictability.paper_definition_q_acceptance q C  $\wedge$ 
    ∀ {X : Finset α} {x y : α}, x ∈ C X  $\rightarrow$  y ∈ X  $\rightarrow$  y ∉ C X  $\rightarrow$  r x y
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_q_representativeness_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 17

Source locator: cited publication, lines 654-657

Verbatim source input

```
\begin{restatable}[$0$-Instability]{definition}{zerostability}
\label{def:zero-instability}
Choice function $\text{\choice}$ is 0-unstable if and only if $\text{\textit{r\_choice}}(X_1, X_2) = 0$ for all $X_1 \text{\textit{subseteq}} X_2$.
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_zero_instability C ↔
  ∀ {X₁ X₂ : Finset α}, X₁ ⊆ X₂ → EconCSLib.FiniteChoice.choiceDistance C X₁ X₂ = 0
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_zero_instability_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 18

Source locator: cited publication, lines 671-679

Verbatim source input

```
\begin{restatable}{definition}{Independence}
\label{def:independence}
$\choice$ is independent if for
  for all $x$,
  either $x \in \choice(X)$
  or $x \notin \choice(X)$ for all $X$ where $x \in X$.
\end{restatable}
```

In other words, any element is either always accepted, or always rejected, when available, irrespective of the other elements
↪ available in a set S .

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_independence C ↔
    ∀ (x : α), (∀ (X : Finset α), x ∈ X → x ∈ C X) ∨ ∀ (X : Finset α), x ∈ X → x ∉ C X
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_independence_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 19

Source locator: cited publication, lines 826-830

Verbatim source input

As per \Cref{def:variability}, when appending a new element to the input set X , different applicants may become rejected: let $\hookrightarrow$ the set of ``borderline admits'' $V_{\text{choice}(X)}$ be

$$V_{\text{choice}(X)} = \bigcup \limits_{x' \in \text{cal} X} \text{choice}(X) \setminus \text{choice}(X \cup \{x'\})$$

which we shall call the ``borderline set'' for short. Notice that this is bounded by q , as $|\text{choice}(X)| = q$.

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α)
  (X : Finset α),
  DGD26AdmissionsPredictability.paper_borderline_set C X = Finset.univ.biUnion fun x => C X \ C (insert x X)
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_definition_borderline_set

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 20

Source locator: cited publication, lines 832-836

Verbatim source input

Conversely, when removing an applicant from X , let the set of applicants that might be newly accepted be

$$A \setminus \text{choice}(X) = \bigcup_{x^* \in X} \text{choice}(X \setminus \{x^*\}) \setminus \text{choice}(X)$$

the set of ``waitlisted rejections''; likewise, we shall call this the ``waitlisted set''.

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α)
  (X : Finset α), DGD26AdmissionsPredictability.paper_waitlisted_set C X = X.biUnion fun x => C (X.erase x) \ C X
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_definition_waitlisted_set

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 21

Source locator: cited publication, lines 838-842

Verbatim source input

```
\begin{definition}[Variability, General]
\label{def:variability-general}
A choice function  $\text{\textit{\textit{choice}}}$  has  $\text{\textit{\textit{variability}}}$   $m$  where:

$$m := \max\left\{\max_{X \subset \mathcal{A}X} \left|V_{\text{\textit{\textit{choice}}}(X)}\right|, \max_{X \subset \mathcal{A}X} \left|A_{\text{\textit{\textit{choice}}}(X)}\right| \right\}$$

\end{definition}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] (m : ℕ)
(C : DGD26AdmissionsPredictability.PaperChoiceRule α),
DGD26AdmissionsPredictability.paper_definition_general_variability_exactly m C ↔
DGD26AdmissionsPredictability.paper_definition_general_variability_at_most m C ∧
((∃ X, (DGD26AdmissionsPredictability.paper_definition_borderline_set C X).card = m) ∨
 ∃ X, (DGD26AdmissionsPredictability.paper_definition_waitlisted_set C X).card = m)
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_general_variability_exactly_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 22

Source locator: cited publication, lines 1073-1084

Verbatim source input

`\subsubsection{Linear Assignment Problems} \label{sec:lap-proofs}`

We highlight `\textit{linear assignment problems}` as a generalization of sequential composition.

The linear sum assignment problem can be represented as a bipartite graph matching.

One side of the graph are applicants, and on the other are available admissions slots.

Edges connect these nodes according to how well suited each applicant is for each slot.

The admissions decision is made by selecting edges which have maximal sum, subject to the constraint that each applicant and each slot is selected only once.

Consider all edges connecting to slot s ; then for each element $x_j \in X$ there is a real number weight $w(x_j, s)$. Then

$\hookrightarrow$ sorting $x \in X$ by $w(x, s)$ creates a total ordering, which we call succ_s . Let slot s' be `\textit{equivalent}` to slot s if and only if $\text{succ}_{s'} = \text{succ}_s$.

We will assume for simplicity that weights are defined such that ties do not occur.

Let assignment A be the set of pairs $\{(x_{s_1}, s_1), \dots, (x_{s_q}, s_q)\}$.

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {σ : Type u_2} [inst_1 : DecidableEq σ] [inst_2 : Fintype σ] (X : Finset α)
(w : α → σ → ℝ) (A : DGD26AdmissionsPredictability.LAP.Assignment α σ),
  DGD26AdmissionsPredictability.lapModel X w A  $\leftrightarrow$ 
    DGD26AdmissionsPredictability.paper_definition_lap_assignment_feasible X A  $\wedge$ 
    DGD26AdmissionsPredictability.paper_definition_lap_capacity_filling X A  $\wedge$ 
    DGD26AdmissionsPredictability.paper_definition_lap_objective_optimal X w A
```

Lean proof endpoint (built separately)

`DGD26AdmissionsPredictability.PaperInterface.paper_lap_model_definition_statement`

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 23

Source locator: cited publication, lines 1081-1083

Verbatim source input

Consider all edges connecting to slot s ; then for each element $x_j \in X$ there is a real number weight $w(x_j, s)$. Then $\hookrightarrow$ sorting $x \in \mathcal{X}$ by $w(x, s)$ creates a total ordering, which we call succ_s . Let slot s' be *equivalent* to slot s if and only if $\text{succ}_{s'} = \text{succ}_s$. We will assume for simplicity that weights are defined such that ties do not occur.

Expanded PaperInterface specification

$\forall \{\alpha : \text{Type } u_1\} \{\sigma : \text{Type } u_2\} [\text{inst} : \text{DecidableEq } \sigma] [\text{inst}_1 : \text{Fintype } \sigma] (w : \alpha \rightarrow \sigma \rightarrow \mathbb{R}) (s : \sigma) (x \ y : \alpha),$
DGD26AdmissionsPredictability.paper_definition_lap_slot_below $w \ s \ x \ y \leftrightarrow w \ x \ s < w \ y \ s$

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_lap_slot_below_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 24

Source locator: cited publication, lines 1081-1083

Verbatim source input

Consider all edges connecting to slot s ; then for each element $x_j \in X$ there is a real number weight $w(x_j, s)$. Then $\hookrightarrow$ sorting $x \in \mathcal{X}$ by $w(x, s)$ creates a total ordering, which we call succ_s . Let slot s' be *equivalent* to slot s if and only if $\text{succ}_{s'} = \text{succ}_s$. We will assume for simplicity that weights are defined such that ties do not occur.

Expanded PaperInterface specification

$\forall \{\alpha : \text{Type } u_1\} \{\sigma : \text{Type } u_2\} [\text{DecidableEq } \sigma] [\text{Fintype } \sigma] (w : \alpha \rightarrow \sigma \rightarrow \mathbb{R}) (s \ t : \sigma),$
DGD26AdmissionsPredictability.LAP.Assignment.SameSlotOrder $w \ s \ t \leftrightarrow \forall (x \ y : \alpha), w \ x \ s < w \ y \ s \leftrightarrow w \ x \ t < w \ y \ t$

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_lap_same_slot_order_definition_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 25

Source locator: cited publication, lines 1084-1089

Verbatim source input

Let assignment A be the set of pairs $\{(x_{s_1}, s_1), \dots, (x_{s_q}, s_q)\}$.

$\begin{array}{l} \text{\texttt{\textbackslash begin\{restatable\}\{lemma\}\{LAP Ordering\}}} \\ \text{\texttt{\textbackslash label\{lem:lap-ordering\}}} \end{array}$

Let x_{s_i} denote the element of $\text{choice}(X)$ assigned to slot s_i . Then for all $x \in X$, $x \in \text{choice}(X)$ if $x \text{ succ}_s \hookrightarrow x_{s_i}$, and $x_{s_i} \text{ succ}_s x$ if $x \notin \text{choice}(X)$.

$\text{\texttt{\textbackslash end\{restatable\}}}$

Expanded PaperInterface specification

$\forall \{\alpha : \text{Type } u_1\} [\text{inst} : \text{DecidableEq } \alpha] \{\sigma : \text{Type } u_2\} [\text{inst}_1 : \text{DecidableEq } \sigma] [\text{inst}_2 : \text{Fintype } \sigma]$
(select : $\text{Finset } \alpha \rightarrow \text{DGD26AdmissionsPredictability.LAP.Assignment } \alpha \sigma$) (X : $\text{Finset } \alpha$) (x : α),
x $\in \text{DGD26AdmissionsPredictability.paper_definition_lap_choice_rule select X} \leftrightarrow$
DGD26AdmissionsPredictability.paper_definition_lap_assigned (select X) x

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_lap_assignment_choice_formula_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 26

Source locator: cited publication, lines 704-707

Verbatim source input

```
\begin{restatable}{corollary}{Only Independent Rules are 0-unstable}
\label{thm:independent-zero-unstable}
$\textit{choice}$ is 0-unstable if and only if it is independent, and therefore no $\textit{q}$-acceptant $\textit{choice}$ can be 0-unstable.
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  (DGD26AdmissionsPredictability.paper_definition_independence C ↔
   DGD26AdmissionsPredictability.paper_definition_zero_instability C)
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_independent_zero_unstable_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 27

Source locator: cited publication, lines 704-707

Verbatim source input

```
\begin{restatable}{corollary}{Only Independent Rules are  $\emptyset$ -unstable}
\label{thm:independent-zero-unstable}
 $\text{\choice\$}$  is  $\emptyset$ -unstable if and only if it is independent, and therefore no  $q$ -acceptant  $\text{\choice\$}$  can be  $\emptyset$ -unstable.
\end{restatable}
```

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

For a feasible q -acceptant choice rule C on a universe with an input pool strictly larger than positive capacity q , C is
 $\hookrightarrow$ $\emptyset$ -unstable if and only if it is independent, and C is not $\emptyset$ -unstable.

Archival source anchor: cited publication, lines 704-707

Recorded basis DGD source clarification: the no-zero conclusion is read on the positive-capacity, nontrivial applicant domain.

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
  0 < q →
  ∀ {U : Finset α},
    q < U.card →
    (DGD26AdmissionsPredictability.paper_definition_zero_instability C ↔
     DGD26AdmissionsPredictability.paper_definition_independence C) ∧
    ¬DGD26AdmissionsPredictability.paper_definition_zero_instability C
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_independent_zero_unstable_corollary_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 28

Source locator: cited publication, lines 796-801

Verbatim source input

```
\begin{restatable}{corollary}{No Consistent Tightly Even Instability}
\label{cor:even-inconsistency}
There is no  $q$ -acceptant and consistent  $\text{\choice}$  that is tightly  $d$ -unstable for even values of  $d$ .
\end{restatable}
```

This follows directly from $\text{\Cref{thm:even-inconsistency}}$.

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

For every positive integer k , no q -acceptant consistent choice rule is tightly $2k$ -unstable.

Archival source anchor: cited publication, lines 796-801

Recorded basis DGD source clarification: tightly-even instability is parameterized by $d=2k$ for positive k .

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q k : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
  0 < k →
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
  DGD26AdmissionsPredictability.paper_definition_consistency C →
  ¬DGD26AdmissionsPredictability.paper_definition_tight_d_instability (2 * k) C
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_no_consistent_positive_tightly_even_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 29

Source locator: cited publication, lines 661-667

Verbatim source input

```
\begin{restatable}{corollary}{Zero-Distance}
\label{thm:sub-mon-zero}
Choice function  $\text{\textit{choice}}$  is 0-unstable if and only if it is both substitutable and monotonic.
\end{restatable}
```

This immediately follows from the two terms comprising $\text{\textit{choice}}$ mapping onto substitutability and monotonicity.

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_zero_instability C ↔
  DGD26AdmissionsPredictability.paper_definition_substitutability C ∧
  DGD26AdmissionsPredictability.paper_definition_monotonicity C
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_zero_distance_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 30

Source locator: cited publication, lines 746-765

Verbatim source input

```
\begin{restatable}[Calculating Instability]{lemma}{stablecalc}
\label{lem:unstable-calc}
For all  $q$ -acceptant  $\mathcal{X}$ , let  $X' = X \cup \mathcal{X}$  for  $|X| \geq q$  and  $n = |\mathcal{X} \setminus \mathcal{X}'|$ . Then
 $r_{\mathcal{X}}(X, X') = 2n - \mathbf{1}_{\mathcal{X} \in \mathcal{X}'}$ .
\end{restatable}

\begin{proof}
\textbf{Informal}
If input sets differ by one element, but the selected sets differ by  $n$  elements, then it must be that  $n$  previously accepted
 $\hookrightarrow$  elements were rejected and there must be  $n$  new accepted elements, at least  $n-1$  must have been previously rejected,
 $\hookrightarrow$  because the size of the accepted set is fixed.

\textbf{Formal}
By  $q$ -acceptance,  $|\mathcal{X}| = |\mathcal{X}'| = q$ . Then  $|\mathcal{X} \setminus \mathcal{X}'| = |\mathcal{X}' \setminus \mathcal{X}|$ .
 $\hookrightarrow |\mathcal{X}| = q - |\mathcal{X} \cap \mathcal{X}'|$ .
Second, note that  $|\mathcal{X}' \setminus \mathcal{X}| = |\mathcal{X}' \setminus \mathcal{X}| \cap X \setminus \mathcal{X}$  if  $x' \notin \mathcal{X}$ , and
 $\hookrightarrow$  otherwise  $|\mathcal{X}' \setminus \mathcal{X}| = \mathcal{X}' \cap \left( \mathcal{X}' \cap X \setminus \mathcal{X} \right)$ .
Putting the previous statements together, we get two cases:
\begin{itemize}
\item  $x' \notin \mathcal{X}$ :  $|\mathcal{X} \setminus \mathcal{X}'| = |\mathcal{X}' \setminus \mathcal{X}| = |X \cap \mathcal{X}' \setminus \mathcal{X}|$ 
 $\hookrightarrow |\mathcal{X}' \setminus \mathcal{X}| = |X \cap \mathcal{X}'|$ 
\item  $x' \in \mathcal{X}$ :  $|\mathcal{X} \setminus \mathcal{X}'| = |\mathcal{X}' \setminus \mathcal{X}| = |X \cap \mathcal{X}'| + 1$ 
 $\hookrightarrow |\mathcal{X}' \setminus \mathcal{X}| = |X \cap \mathcal{X}'| + 1$ 
\end{itemize}
Then  $r_{\mathcal{X}}(X, X') = |X \cap \mathcal{X}' \setminus \mathcal{X}| + |\mathcal{X} \setminus \mathcal{X}'| = 2|X \cap \mathcal{X}'|$ 
 $\hookrightarrow |\mathcal{X}' \setminus \mathcal{X}| + \mathbf{1}_{\mathcal{X} \in \mathcal{X}'} = 2|\mathcal{X} \setminus \mathcal{X}'| - \mathbf{1}_{\mathcal{X} \in \mathcal{X}'}$ 
\end{proof}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
  ∀ {X : Finset α} {x : α},
    q ≤ X.card →
    x ∉ X →
    DGD26AdmissionsPredictability.paper_definition_choice_distance C X (insert x X) =
      if x ∈ C (insert x X) then 2 * (C X \ C (insert x X)).card - 1 else 2 * (C X \ C (insert x X)).card
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_calculating_instability_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 31

Source locator: cited publication, lines 885-896

Verbatim source input

```
\begin{restatable}[Consistency of Removable Sets]{lemma}{consistentremove}
\label{lem:consistent-remove}
Let  $\text{\textit{choice}}$  be 1-unstable and  $q$ -acceptant with variability  $\$m$ . For any  $X_1, X_2$ , if  $\text{\textit{choice}}(X_1) = \text{\textit{choice}}(X_2)$ , then
 $\hookrightarrow V_{\text{\textit{choice}}(X_1)} = V_{\text{\textit{choice}}(X_2)}$ .
\end{restatable}

\begin{proof}
Recall from \Cref{thm:sub-accept-consistent} that  $\text{\textit{choice}}$  must be consistent and so  $\text{\textit{choice}}(\text{\textit{choice}}(X)) = \text{\textit{choice}}(X)$  for any
 $\hookrightarrow X$ .

Consider any  $x$  associated with an element  $x^*$  of  $V_{\text{\textit{choice}}(X)}$ , such that  $\text{\textit{choice}}(X) \setminus \text{\textit{choice}}(X') = \{x^*\}$  (and
 $\hookrightarrow$  even more precisely, by  $q$ -acceptant substitutability, that  $\text{\textit{choice}}(X') = \{x'\} \cup \text{\textit{choice}}(X) \setminus \{x^*\}$ 
 $\hookrightarrow$  exactly). Note that  $\text{\textit{choice}}(X') \subseteq \text{\textit{choice}}(X) \cup \{x'\} \subseteq X$ , and therefore  $\text{\textit{choice}}(\text{\textit{choice}}(X) \cup \{x'\}) = \text{\textit{choice}}(X')$  by consistency. This applies to all possible  $x$  (including when  $x \in X$ ), and therefore,
 $\hookrightarrow V_{\text{\textit{choice}}(X)} = V_{\text{\textit{choice}}(\text{\textit{choice}}(X))}$ .

As this set equality is exact, apply this logic to some  $X_2$  and see  $V_{\text{\textit{choice}}(X_1)} = V_{\text{\textit{choice}}(\text{\textit{choice}}(X_1))} =$ 
 $\hookrightarrow V_{\text{\textit{choice}}(\text{\textit{choice}}(X_2))} = V_{\text{\textit{choice}}(X_2)}$ .
\end{proof}
```

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

For a feasible q -acceptant substitutable choice rule C , if $C(X_1)=C(X_2)$, then the borderline sets $V_C(X_1)$ and $V_C(X_2)$ are equal.

Archival source anchor: cited publication, lines 885-896

Recorded basis DGD source clarification: the removable-set equality compares the two pools X_1 and X_2 .

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {q : ℕ}
{C : DGD26AdmissionsPredictability.PaperChoiceRule α},
DGD26AdmissionsPredictability.paper_choice_function_feasible C →
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
DGD26AdmissionsPredictability.paper_definition_substitutability C →
  ∀ {X_1 X_2 : Finset α},
  C X_1 = C X_2 →
    DGD26AdmissionsPredictability.paper_definition_borderline_set C X_1 =
    DGD26AdmissionsPredictability.paper_definition_borderline_set C X_2
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_corrected_consistency_of_removable_sets_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 32

Source locator: cited publication, lines 1086-1095

Verbatim source input

```
\begin{restatable}{lemma}{LAP Ordering}
\label{lem:lap-ordering}
Let  $x_s$  denote the element of  $\text{choice}(X)$  assigned to slot  $s$ . Then for all  $x \in X$ ,  $x \in \text{choice}(X)$  if  $x \text{succ}_s$ 
 $\hookrightarrow x_s$ , and  $x_s \text{succ}_s x$  if  $x \notin \text{choice}(X)$ .
\end{restatable}

\textit{Intuition}: Linear assignment problems have a structure that resembles a combination of rankings, even though there is
 $\hookrightarrow$  not a fixed sequential order.

\begin{proof}
This comes by the optimality of the assignment solution. For contradiction, say there is  $x^* \text{succ}_s x_s$  where  $x^* \notin \text{choice}(X)$ . Then  $w(x^*, s) > w(x_s, s)$ , and therefore  $\{x^*, s\} \cup A \setminus \{x_s, s\}$  leads to a higher sum than
 $\hookrightarrow \text{choice}(X)$ , contradicting the fact that  $A$  is optimal.
\end{proof}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {σ : Type u_2} [inst_1 : DecidableEq σ] [inst_2 : Fintype σ] {X : Finset α}
{w : α → σ → ℝ} {A : DGD26AdmissionsPredictability.LAP.Assignment α σ},
DGD26AdmissionsPredictability.paper_definition_lap_objective_optimal X w A →
DGD26AdmissionsPredictability.paper_definition_lap_capacity_filling X A →
DGD26AdmissionsPredictability.paper_definition_lap_assignment_feasible X A →
  ∀ {s : σ} {y x : α},
    A.matchSlot s = some y →
      x ∈ X →
        DGD26AdmissionsPredictability.paper_definition_lap_slot_below w s y x →
          DGD26AdmissionsPredictability.paper_definition_lap_assigned A x
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_lap_strictly_higher_slot_applicant_assigned_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 33

Source locator: cited publication, lines 1086-1095

Verbatim source input

```
\begin{restatable}{lemma}{LAP Ordering}
\label{lem:lap-ordering}
Let  $x_s$  denote the element of  $\text{choice}(X)$  assigned to slot  $s$ . Then for all  $x \in X$ ,  $x \in \text{choice}(X)$  if  $x \text{ succ}_s$ 
 $\hookrightarrow x_s$ , and  $x_s \text{ succ}_s x$  if  $x \notin \text{choice}(X)$ .
\end{restatable}

\textit{Intuition}: Linear assignment problems have a structure that resembles a combination of rankings, even though there is
 $\hookrightarrow$  not a fixed sequential order.

\begin{proof}
This comes by the optimality of the assignment solution. For contradiction, say there is  $x^* \text{ succ}_s x_s$  where  $x^* \notin \text{choice}(X)$ . Then  $w(x^*, s) > w(x_s, s)$ , and therefore  $\{x^*, s\} \cup A \setminus \{x_s, s\}$  leads to a higher sum than
 $\hookrightarrow \text{choice}(X)$ , contradicting the fact that  $A$  is optimal.
\end{proof}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {σ : Type u_2} [inst_1 : DecidableEq σ] [inst_2 : Fintype σ] {X : Finset α}
{w : α → σ → ℝ} {A : DGD26AdmissionsPredictability.LAP.Assignment α σ},
DGD26AdmissionsPredictability.paper_definition_lap_objective_optimal X w A →
DGD26AdmissionsPredictability.paper_definition_lap_capacity_filling X A →
DGD26AdmissionsPredictability.paper_definition_lap_assignment_feasible X A →
∀ {s : σ} {y x : α},
A.matchSlot s = some y →
DGD26AdmissionsPredictability.LAP.Assignment.Rejected X A x →
DGD26AdmissionsPredictability.LAP.Assignment.SlotNoTies w s →
DGD26AdmissionsPredictability.paper_definition_lap_slot_below w s x y
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_lap_assigned_strictly_outranks_rejected_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 34

Source locator: cited publication, lines 564-573

Verbatim source input

```
\begin{restatable}[Monotonicity Term]{lemma}{mono-dist}
\label{lem:monotonicity-term}
Choice function  $\text{\choice}$  is monotonic if and only if  $|\text{\choice}(X_1) \setminus \text{\choice}(X_2)| = 0$  for all  $X_1 \subseteq X_2$ .
\end{restatable}

\textit{Intuition}:
For a monotonic choice function, previously accepted candidates are never rejected after the addition of more applicants. Thus
 $\hookrightarrow$  labels cannot flip from positive to negative, which is what the second term counts.

\begin{proof}
Use the same set properties as per \Cref{lem:substitutability-term},  $|\text{\choice}(X_1) \setminus \text{\choice}(X_2)| = 0$  if and only if
 $\hookrightarrow \text{\choice}(X_1) \subseteq \text{\choice}(X_2)$ , which is the definition of monotonicity.
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_monotonicity C ↔ ∀ {X₁ X₂ : Finset α}, X₁ ⊆ X₂ → (C X₁ \ C X₂).card = 0
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_monotonicity_term_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 35

Source locator: cited publication, lines 517-529

Verbatim source input

```
\begin{restatable}[Incompatibility of Monotonicity and  $q$ -Acceptance]{lemma}{incompatibility}
\label{thm:mono-accept-mutex}
For  $|\mathcal{X}| > q \geq 1$ , no choice function can be both monotonic and  $q$ -acceptant.
\end{restatable}

\textit{Intuition}: %
For a choice function to be monotonic, it must never reject a previously accepted applicant as the applicant pool grows. This
 $\hookrightarrow$  implies that the capacity must be unbounded, which contradicts the strict capacity limit imposed by  $q$ -acceptance.%

\begin{proof}
Consider two different sets each of  $q$  elements. If  $\text{choice}$  is  $q$ -acceptant, every element must be chosen in each. Then
 $\hookrightarrow$  consider their union. If  $\text{choice}$  is monotonic, every element in their union would also be accepted, but that would
 $\hookrightarrow$  contradict the  $q$ -acceptant capacity constraint.

Consider two disjoint sets  $X_1, X_2 \subseteq \mathcal{X}$  with  $|X_1| = |X_2| = q$ . By  $q$ -acceptance,  $\text{choice}(X_1) = X_1$  and
 $\hookrightarrow \text{choice}(X_2) = X_2$ . By monotonicity, both  $\text{choice}(X_1)$  and  $\text{choice}(X_2)$  must be subsets of  $\text{choice}(X_1 \cup X_2)$ . But
 $\hookrightarrow$  then  $|\text{choice}(X_1 \cup X_2)| \geq |X_1| + |X_2| = 2q > q$ , violating  $q$ -acceptance.
\end{proof}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_monotonicity C →
  DGD26AdmissionsPredictability.paper_definition_q_acceptance q C → 0 < q → ∀ {U : Finset α}, q < U.card → False
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_monotonicity_q_acceptance_incompatible_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 36

Source locator: cited publication, lines 898-905

Verbatim source input

```
\begin{restatable}{lemma}{Ranking m}
\label{lem:a-v-equal}
Let  $q$ -acceptant,  $1$ -unstable  $\mathcal{X}$  have variability  $1$ . Then  $V_{\mathcal{X}} = A_{\mathcal{X}} \cup \{x'\}$  when  $\mathcal{X}$ 
 $\hookrightarrow \mathcal{X} \cup \{x'\} \neq \mathcal{X}$ .
\end{restatable}

\begin{proof}
If  $\mathcal{X} \cup \{x'\} \neq \mathcal{X}$ , then  $\mathcal{X} \cup \{x'\} = \mathcal{X} \setminus x^*$  where
 $\hookrightarrow V_{\mathcal{X}} = \mathcal{X} \setminus x^*$  (by consistency, 1-instability, and the definition of variability). Then  $\mathcal{X}' \setminus x' = \mathcal{X}$ 
 $\hookrightarrow$  and so  $A_{\mathcal{X}'} = \mathcal{X} \setminus x^*$ , completing the proof.
\end{proof}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {q : ℕ}
{C : DGD26AdmissionsPredictability.PaperChoiceRule α},
DGD26AdmissionsPredictability.paper_choice_function_feasible C →
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
DGD26AdmissionsPredictability.paper_definition_d_instability 1 C →
DGD26AdmissionsPredictability.paper_definition_variability_at_most 1 C →
∀ {X : Finset α} {x : α},
x ∉ X →
C (insert x X) ≠ C X →
DGD26AdmissionsPredictability.paper_definition_borderline_set C X =
DGD26AdmissionsPredictability.paper_definition_waitlisted_set C (insert x X)
```

Lean proof endpoint (built separately)

```
DGD26AdmissionsPredictability.PaperInterface.paper_acceptant_one_instability_variability_borderline_eq_waitlisted_after_changing_
 $\hookrightarrow$  insert_statement
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 37

Source locator: cited publication, lines 552-562

Verbatim source input

```
\begin{restatable}[Substitutability Term]{lemma}{sub-dist}
\label{lem:substitutability-term}
Choice function  $\text{\choice}$  is substitutable if and only if  $|X_1 \cap \text{\choice}(X_2) \setminus \text{\choice}(X_1)| = 0$  for all  $X_1$ 
 $\hookrightarrow \text{\subseq} X_2$ .
\end{restatable}

\textit{Intuition}:
For a substitutable choice function, previously rejected candidates are never accepted after the addition of more applicants.
 $\hookrightarrow$  Thus labels cannot flip from negative to positive, which is what the first term counts.

\begin{proof}
For any sets  $A, B$ ,  $|A \setminus B| = 0$  if and only if  $A \setminus B = \emptyset$ , and furthermore  $A \setminus B = \emptyset$  if and only if  $A \subseteq B$ . So  $|X_1 \cap \text{\choice}(X_2) \setminus \text{\choice}(X_1)| = 0$  if and only if  $X_1 \cap \text{\choice}(X_2) \subseteq \text{\choice}(X_1)$ , which is the definition of substitutability.
\end{proof}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  DGD26AdmissionsPredictability.paper_definition_substitutability C ↔
    ∀ {X₁ X₂ : Finset α}, X₁ ⊆ X₂ → ((X₁ ∩ C X₂) \ C X₁).card = 0
```

Lean proof endpoint (built separately)

```
DGD26AdmissionsPredictability.PaperInterface.paper_substitutability_term_statement
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 38

Source locator: cited publication, lines 532-546

Verbatim source input

```
\begin{restatable}{lemma}{unsubstitutable}
\label{lem:unsubstitutable}
If  $\text{\choice}$  is not substitutable, there exists some  $x^*$ ,  $x'$ ,  $X$  and  $X' = X \cup \{x'\}$  where  $x^* \in X$  and  $x^* \in \text{\choice}(X')$  but  $x^* \notin \text{\choice}(X)$ .
\end{restatable}

\textit{Intuition}:
If a choice function is not substitutable, then there is at least one situation where a decision ``flips'' from rejection to acceptance.
We argue that there has to be some single element  $x'$  that causes this ``flip'' when added.

\begin{proof}
 $\text{\choice}$  is not substitutable if there exists some  $x^*$  and  $X_0 \subseteq X_k$  where  $x^* \in X_0$  and  $x^* \in \text{\choice}(X_k)$  but  $x^* \notin \text{\choice}(X_0)$ .
Let  $k$  be such that  $k = |X_k \setminus X_0|$ .

We want to show the existence of  $x'$ . To do this, we identify the ``closest'' sets where this occurs. For each  $x \in X_k \setminus X_0$ , add elements to  $X_0$ , constructing  $X_0 \subseteq X_1 \subseteq X_2 \subseteq \dots \subseteq X_j \subseteq X_{j+1} \subseteq \dots \subseteq X_k$ , where  $X_1 = X_0 \cup \{x_1\}$  and so on. Let  $X_j$  be the first in this sequence of sets such that  $x^* \in \text{\choice}(X_j)$  where  $x^* \in \text{\choice}(X_j)$  but  $x^* \notin \text{\choice}(X_{j-1})$ . Then let  $x' = x_j$ ,  $X = X_{j-1}$ , and  $X' = X_j$ .
\end{proof}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α),
  ¬DGD26AdmissionsPredictability.paper_definition_substitutability C →
    ∃ X x xstar, x ∉ X ∧ xstar ∈ X ∧ xstar ∈ C (insert x X) ∧ xstar ∉ C X
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_non_substitutable_single_add_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 39

Source locator: cited publication, lines 261-265

Verbatim source input

```
\begin{restatable}[ML Representation]{proposition}{mlrepresentation}
\label{prop:ml-representation}
A model of the form~\eqref{eq:ml-independent} which makes independent predictions can only represent 0-unstable choice functions.
A model of the form~\eqref{eq:ml-rank} which ranks applicants can represent a 1-variable, 1-unstable choice function. No such
↪ model can represent a choice function with variability or instability greater than one.
\end{restatable}
```

[Next verbatim source excerpt]

```
\subsection{Proof of \Cref{prop:ml-representation}} \label{sec:ml-prop-proof}
```

```
Recall \Cref{prop:ml-representation}:
\mlrepresentation*
```

First consider using an ML model to make decisions; then it can be seen as a choice function with various properties. Thus we say that an ML model can represent a choice function with various properties if it can have those properties when viewed ↪ as a choice function. %

The first statement follows by noting that any rule of the form~\eqref{eq:ml-independent} must either admit or reject an ↪ applicant regardless of the applicant pool, so it must be independent and hence \$0\$-unstable by ↪ \Cref{thm:independent-zero-unstable}. %
The second and third statements follow by noting that scores in~\eqref{eq:ml-rank} induce a total order by embedding applicants ↪ in $\mathbb{R}$ and selecting accordingly. Therefore any rule of the form~\eqref{eq:ml-rank} is \$q\$-representative (by ↪ definition) and hence has instability and variability exactly one (by \Cref{thm:ranking-variability}), but no greater.

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {C : DGD26AdmissionsPredictability.PaperChoiceRule α} (score : α → ℝ)
(threshold : ℝ),
DGD26AdmissionsPredictability.paper_choice_function_feasible C →
DGD26AdmissionsPredictability.paper_definition_ml_representation
(DGD26AdmissionsPredictability.paper_definition_fixed_threshold_predictor score threshold) C →
DGD26AdmissionsPredictability.paper_definition_zero_instability C
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_ml_fixed_threshold_representation_zero_unstable_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 40

Source locator: cited publication, lines 261-269

Verbatim source input

```
\begin{restatable}[ML Representation]{proposition}{mlrepresentation}
\label{prop:ml-representation}
A model of the form~\eqref{eq:ml-independent} which makes independent predictions can only represent 0-unstable choice functions.
A model of the form~\eqref{eq:ml-rank} which ranks applicants can represent a 1-variable, 1-unstable choice function. No such
↪ model can represent a choice function with variability or instability greater than one.
\end{restatable}
```

This result shows that standard ML practice coherently applies to admissions data only in limited settings. The intuition for this result hinges on the fact that all 1-unstable and 1-variable choice functions correspond to total preference orderings over applicants (formalized in Theorem~\ref{thm:variability}). Then the proposition follows from noticing that total preference orderings correspond to embedding applicants in $\mathbb{R}^d$ (by their scores).

[Next verbatim source excerpt]

```
\subsection{Proof of \Cref{prop:ml-representation}} \label{sec:ml-prop-proof}
```

Recall \Cref{prop:ml-representation}:
\mlrepresentation*

First consider using an ML model to make decisions; then it can be seen as a choice function with various properties. Thus we say that an ML model can represent a choice function with various properties if it can have those properties when viewed as a choice function. %

The first statement follows by noting that any rule of the form~\eqref{eq:ml-independent} must either admit or reject an applicant regardless of the applicant pool, so it must be independent and hence 0-unstable by \Cref{thm:independent-zero-unstable}. %
The second and third statements follow by noting that scores in~\eqref{eq:ml-rank} induce a total order by embedding applicants in $\mathbb{R}^d$ and selecting accordingly. Therefore any rule of the form~\eqref{eq:ml-rank} is q -representative (by definition) and hence has instability and variability exactly one (by \Cref{thm:ranking-variability}), but no greater.

Expanded PaperInterface specification

```
∀ (q : ℕ),
  0 < q →
    ∃ score,
      ∃ (hinjective : Function.Injective score),
        DGD26AdmissionsPredictability.paper_definition_tight_d_instability 1
          (DGD26AdmissionsPredictability.paperRankThresholdChoice q score hinjective) ∧
          DGD26AdmissionsPredictability.paper_definition_variability_exactly 1
            (DGD26AdmissionsPredictability.paperRankThresholdChoice q score hinjective)
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_ml_rank_threshold_can_represent_exact_one_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 41

Source locator: cited publication, lines 261-269

Verbatim source input

```
\begin{restatable}[ML Representation]{proposition}{mlrepresentation}
\label{prop:ml-representation}
A model of the form~\eqref{eq:ml-independent} which makes independent predictions can only represent 0-unstable choice functions.
A model of the form~\eqref{eq:ml-rank} which ranks applicants can represent a 1-variable, 1-unstable choice function. No such
↪ model can represent a choice function with variability or instability greater than one.
\end{restatable}
```

This result shows that standard ML practice coherently applies to admissions data only in limited settings. The intuition for this result hinges on the fact that all 1-unstable and 1-variable choice functions correspond to total preference orderings over applicants (formalized in Theorem~\ref{thm:variability}). Then the proposition follows from noticing that total preference orderings correspond to embedding applicants in $\mathbb{R}^S$ (by their scores).

[Next verbatim source excerpt]

```
\subsection{Proof of \Cref{prop:ml-representation}} \label{sec:ml-prop-proof}
```

Recall \Cref{prop:ml-representation}:
\mlrepresentation*

First consider using an ML model to make decisions; then it can be seen as a choice function with various properties. Thus we say that an ML model can represent a choice function with various properties if it can have those properties when viewed as a choice function. %

The first statement follows by noting that any rule of the form~\eqref{eq:ml-independent} must either admit or reject an applicant regardless of the applicant pool, so it must be independent and hence 0-unstable by \Cref{thm:independent-zero-unstable}. %
The second and third statements follow by noting that scores in~\eqref{eq:ml-rank} induce a total order by embedding applicants in $\mathbb{R}^S$ and selecting accordingly. Therefore any rule of the form~\eqref{eq:ml-rank} is q -representative (by definition) and hence has instability and variability exactly one (by \Cref{thm:ranking-variability}), but no greater.

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [Fintype α] {C : DGD26AdmissionsPredictability.PaperChoiceRule α} (q : ℕ)
  (score : α → ℝ) (hinjective : Function.Injective score),
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_ml_representation
    (DGD26AdmissionsPredictability.paper_definition_rank_threshold_predictor q score hinjective) C →
  DGD26AdmissionsPredictability.paper_definition_d_instability 1 C
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_ml_rank_threshold_representation_instability_bound_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 42

Source locator: cited publication, lines 261-269

Verbatim source input

```
\begin{restatable}[ML Representation]{proposition}{mlrepresentation}
\label{prop:ml-representation}
A model of the form~\eqref{eq:ml-independent} which makes independent predictions can only represent 0-unstable choice functions.
A model of the form~\eqref{eq:ml-rank} which ranks applicants can represent a 1-variable, 1-unstable choice function. No such
↪ model can represent a choice function with variability or instability greater than one.
\end{restatable}
```

This result shows that standard ML practice coherently applies to admissions data only in limited settings. The intuition for this result hinges on the fact that all 1-unstable and 1-variable choice functions correspond to total preference orderings over applicants (formalized in Theorem~\ref{thm:variability}). Then the proposition follows from noticing that total preference orderings correspond to embedding applicants in $\mathbb{R}^S$ (by their scores).

[Next verbatim source excerpt]

```
\subsection{Proof of \Cref{prop:ml-representation}} \label{sec:ml-prop-proof}
```

Recall \Cref{prop:ml-representation}:
\mlrepresentation*

First consider using an ML model to make decisions; then it can be seen as a choice function with various properties. Thus we say that an ML model can represent a choice function with various properties if it can have those properties when viewed as a choice function. %

The first statement follows by noting that any rule of the form~\eqref{eq:ml-independent} must either admit or reject an applicant regardless of the applicant pool, so it must be independent and hence 0-unstable by \Cref{thm:independent-zero-unstable}. %
The second and third statements follow by noting that scores in~\eqref{eq:ml-rank} induce a total order by embedding applicants in $\mathbb{R}^S$ and selecting accordingly. Therefore any rule of the form~\eqref{eq:ml-rank} is q -representative (by definition) and hence has instability and variability exactly one (by \Cref{thm:ranking-variability}), but no greater.

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {C : DGD26AdmissionsPredictability.PaperChoiceRule α}
  (q : ℕ) (score : α → ℝ) (hinjective : Function.Injective score),
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_ml_representation
    (DGD26AdmissionsPredictability.paper_definition_rank_threshold_predictor q score hinjective) C →
  DGD26AdmissionsPredictability.paper_definition_variability_at_most 1 C
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_ml_rank_threshold_representation_variability_bound_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 43

Source locator: cited publication, lines 347-358

Verbatim source input

```
\begin{restatable}[]{}{proposition}{empiricalmethods}
\label{prop:program-variability}
All considered choice functions ({Ed. Opt}, {Screened/Open}, {Ed. Opt with DIA}, and {Screened/Open with DIA}) are $1$-unstable.
↪ Furthermore, their variability is (tightly) equal to their number of queues:

\begin{itemize}[noitemsep,topsep=0pt,parsep=0pt,partopsep=0pt]
  \item Screened/Open programs are $1$-variable
  \item Screened/Open with DIA programs are $2$-variable
  \item Ed. Opt. programs are $3$-variable
  \item Ed. Opt with DIA programs are $6$-variable
\end{itemize}

\end{restatable}

[Next verbatim source excerpt]

\subsection{Proof of \Cref{prop:program-variability}} \label{sec:program-prop-proof}

Recall \Cref{prop:program-variability}:
\empiricalmethods*

We construct applicant sets to prove the variability of each empirical method we study, showing that the upper bound from
↪ \Cref{thm:variability} based on the number of queues is tight.

Note that both Screened and Open programs are explicitly a ranking, and therefore have variability $1$ by \Cref{thm:variability}.
Screened and Open programs with DIA are upper-bounded by variability 2, as they are composed of two queues. It is easy to see
↪ that this upper bound is tight. Consider an applicant pool $X$ where the lowest-ranking students in the DIA queue is
↪ lower-ranked than the lowest-ranked student in the non-DIA queue. Then $A \setminus \text{choice}(X)$ contains two elements -- adding a new
↪ DIA-qualified student might lead to rejecting the lowest-ranked student in the DIA queue, while a non-DIA student would lead
↪ to a change in the non-DIA queue. Similarly, in the three-queue Ed. Opt. case, the variability $3$ bound is tight, as there
↪ are applicant pools where adding a single high-, middle-, or low-category student would displace a different student in the
↪ corresponding queue. Likewise, the six queues in Ed. Opt. with DIA programs lead to six different students who might be
↪ displaced depending on what category a new applicant falls into.
```

Expanded PaperInterface specification

```
DGD26AdmissionsPredictability.paper_definition_d_instability 1
  DGD26AdmissionsPredictability.paper_definition_screened_open_program_choice 1
DGD26AdmissionsPredictability.paper_definition_d_instability 1
  DGD26AdmissionsPredictability.paper_definition_screened_open_dia_program_choice 1
DGD26AdmissionsPredictability.paper_definition_d_instability 1
  DGD26AdmissionsPredictability.paper_definition_educational_option_program_choice 1
DGD26AdmissionsPredictability.paper_definition_d_instability 1
  DGD26AdmissionsPredictability.paper_definition_educational_option_dia_program_choice 1
```

Lean proof endpoint (built separately)

```
DGD26AdmissionsPredictability.PaperInterface.paper_program_classes_one_instability_statement
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 44

Source locator: cited publication, lines 347-358

Verbatim source input

```
\begin{restatable}[]{\proposition}{empiricalmethods}
\label{prop:program-variability}
All considered choice functions ({Ed. Opt}, {Screened/Open}, {Ed. Opt with DIA}, and {Screened/Open with DIA}) are $1$-unstable.
↪ Furthermore, their variability is (tightly) equal to their number of queues:

\begin{itemize}[noitemsep,topsep=0pt,parsep=0pt,partopsep=0pt]
  \item Screened/Open programs are $1$-variable
  \item Screened/Open with DIA programs are $2$-variable
  \item Ed. Opt. programs are $3$-variable
  \item Ed. Opt with DIA programs are $6$-variable
\end{itemize}

\end{restatable}

[Next verbatim source excerpt]

\subsection{Proof of \Cref{prop:program-variability}} \label{sec:program-prop-proof}

Recall \Cref{prop:program-variability}:
\empiricalmethods*

We construct applicant sets to prove the variability of each empirical method we study, showing that the upper bound from
↪ \Cref{thm:variability} based on the number of queues is tight.

Note that both Screened and Open programs are explicitly a ranking, and therefore have variability $1$ by \Cref{thm:variability}.
Screened and Open programs with DIA are upper-bounded by variability 2, as they are composed of two queues. It is easy to see
↪ that this upper bound is tight. Consider an applicant pool $X$ where the lowest-ranking students in the DIA queue is
↪ lower-ranked than the lowest-ranked student in the non-DIA queue. Then $A_{\text{choice}(X)}$ contains two elements -- adding a new
↪ DIA-qualified student might lead to rejecting the lowest-ranked student in the DIA queue, while a non-DIA student would lead
↪ to a change in the non-DIA queue. Similarly, in the three-queue Ed. Opt. case, the variability $3$ bound is tight, as there
↪ are applicant pools where adding a single high-, middle-, or low-category student would displace a different student in the
↪ corresponding queue. Likewise, the six queues in Ed. Opt. with DIA programs lead to six different students who might be
↪ displaced depending on what category a new applicant falls into.
```

Expanded PaperInterface specification

```
DGD26AdmissionsPredictability.paper_definition_variability_exactly 1
DGD26AdmissionsPredictability.paper_definition_screened_open_program_choice
```

Lean proof endpoint (built separately)

```
DGD26AdmissionsPredictability.PaperInterface.paper_screened_open_variability_one_statement
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 45

Source locator: cited publication, lines 347-358

Verbatim source input

```
\begin{restatable}[]{\proposition}{empiricalmethods}
\label{prop:program-variability}
All considered choice functions ({Ed. Opt}, {Screened/Open}, {Ed. Opt with DIA}, and {Screened/Open with DIA}) are $1$-unstable.
↪ Furthermore, their variability is (tightly) equal to their number of queues:

\begin{itemize}[noitemsep,topsep=0pt,parsep=0pt,partopsep=0pt]
  \item Screened/Open programs are $1$-variable
  \item Screened/Open with DIA programs are $2$-variable
  \item Ed. Opt. programs are $3$-variable
  \item Ed. Opt with DIA programs are $6$-variable
\end{itemize}

\end{restatable}

[Next verbatim source excerpt]

\subsection{Proof of \Cref{prop:program-variability}} \label{sec:program-prop-proof}

Recall \Cref{prop:program-variability}:
\empiricalmethods*

We construct applicant sets to prove the variability of each empirical method we study, showing that the upper bound from
↪ \Cref{thm:variability} based on the number of queues is tight.

Note that both Screened and Open programs are explicitly a ranking, and therefore have variability $1$ by \Cref{thm:variability}.
Screened and Open programs with DIA are upper-bounded by variability 2, as they are composed of two queues. It is easy to see
↪ that this upper bound is tight. Consider an applicant pool $X$ where the lowest-ranking students in the DIA queue is
↪ lower-ranked than the lowest-ranked student in the non-DIA queue. Then $A_{\choice(X)}$ contains two elements -- adding a new
↪ DIA-qualified student might lead to rejecting the lowest-ranked student in the DIA queue, while a non-DIA student would lead
↪ to a change in the non-DIA queue. Similarly, in the three-queue Ed. Opt. case, the variability $3$ bound is tight, as there
↪ are applicant pools where adding a single high-, middle-, or low-category student would displace a different student in the
↪ corresponding queue. Likewise, the six queues in Ed. Opt. with DIA programs lead to six different students who might be
↪ displaced depending on what category a new applicant falls into.
```

Expanded PaperInterface specification

```
DGD26AdmissionsPredictability.paper_definition_variability_exactly 2
DGD26AdmissionsPredictability.paper_definition_screened_open_dia_program_choice
```

Lean proof endpoint (built separately)

```
DGD26AdmissionsPredictability.PaperInterface.paper_screened_open_dia_variability_two_statement
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 46

Source locator: cited publication, lines 347-358

Verbatim source input

```
\begin{restatable}[]{}{proposition}{empiricalmethods}
\label{prop:program-variability}
All considered choice functions ({Ed. Opt}, {Screened/Open}, {Ed. Opt with DIA}, and {Screened/Open with DIA}) are $1$-unstable.
↪ Furthermore, their variability is (tightly) equal to their number of queues:

\begin{itemize}[noitemsep,topsep=0pt,parsep=0pt,partopsep=0pt]
\item Screened/Open programs are $1$-variable
\item Screened/Open with DIA programs are $2$-variable
\item Ed. Opt. programs are $3$-variable
\item Ed. Opt with DIA programs are $6$-variable
\end{itemize}

\end{restatable}

[Next verbatim source excerpt]

\subsection{Proof of \Cref{prop:program-variability}} \label{sec:program-prop-proof}

Recall \Cref{prop:program-variability}:
\empiricalmethods*

We construct applicant sets to prove the variability of each empirical method we study, showing that the upper bound from
↪ \Cref{thm:variability} based on the number of queues is tight.

Note that both Screened and Open programs are explicitly a ranking, and therefore have variability $1$ by \Cref{thm:variability}.
Screened and Open programs with DIA are upper-bounded by variability 2, as they are composed of two queues. It is easy to see
↪ that this upper bound is tight. Consider an applicant pool $X$ where the lowest-ranking students in the DIA queue is
↪ lower-ranked than the lowest-ranked student in the non-DIA queue. Then $A_{\text{choice}(X)}$ contains two elements -- adding a new
↪ DIA-qualified student might lead to rejecting the lowest-ranked student in the DIA queue, while a non-DIA student would lead
↪ to a change in the non-DIA queue. Similarly, in the three-queue Ed. Opt. case, the variability $3$ bound is tight, as there
↪ are applicant pools where adding a single high-, middle-, or low-category student would displace a different student in the
↪ corresponding queue. Likewise, the six queues in Ed. Opt. with DIA programs lead to six different students who might be
↪ displaced depending on what category a new applicant falls into.
```

Expanded PaperInterface specification

```
DGD26AdmissionsPredictability.paper_definition_variability_exactly 3
DGD26AdmissionsPredictability.paper_definition_educational_option_program_choice
```

Lean proof endpoint (built separately)

```
DGD26AdmissionsPredictability.PaperInterface.paper_educational_option_variability_three_statement
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 47

Source locator: cited publication, lines 347-358

Verbatim source input

```
\begin{restatable}[]{\proposition}{empiricalmethods}
\label{prop:program-variability}
All considered choice functions ({Ed. Opt}, {Screened/Open}, {Ed. Opt with DIA}, and {Screened/Open with DIA}) are $1$-unstable.
↪ Furthermore, their variability is (tightly) equal to their number of queues:

\begin{itemize}[noitemsep,topsep=0pt,parsep=0pt,partopsep=0pt]
  \item Screened/Open programs are $1$-variable
  \item Screened/Open with DIA programs are $2$-variable
  \item Ed. Opt. programs are $3$-variable
  \item Ed. Opt with DIA programs are $6$-variable
\end{itemize}

\end{restatable}

[Next verbatim source excerpt]

\subsection{Proof of \Cref{prop:program-variability}} \label{sec:program-prop-proof}

Recall \Cref{prop:program-variability}:
\empiricalmethods*

We construct applicant sets to prove the variability of each empirical method we study, showing that the upper bound from
↪ \Cref{thm:variability} based on the number of queues is tight.

Note that both Screened and Open programs are explicitly a ranking, and therefore have variability $1$ by \Cref{thm:variability}.
Screened and Open programs with DIA are upper-bounded by variability 2, as they are composed of two queues. It is easy to see
↪ that this upper bound is tight. Consider an applicant pool $X$ where the lowest-ranking students in the DIA queue is
↪ lower-ranked than the lowest-ranked student in the non-DIA queue. Then $A_{\text{choice}(X)}$ contains two elements -- adding a new
↪ DIA-qualified student might lead to rejecting the lowest-ranked student in the DIA queue, while a non-DIA student would lead
↪ to a change in the non-DIA queue. Similarly, in the three-queue Ed. Opt. case, the variability $3$ bound is tight, as there
↪ are applicant pools where adding a single high-, middle-, or low-category student would displace a different student in the
↪ corresponding queue. Likewise, the six queues in Ed. Opt. with DIA programs lead to six different students who might be
↪ displaced depending on what category a new applicant falls into.
```

Expanded PaperInterface specification

```
DGD26AdmissionsPredictability.paper_definition_variability_exactly 6
DGD26AdmissionsPredictability.paper_definition_educational_option_dia_program_choice
```

Lean proof endpoint (built separately)

```
DGD26AdmissionsPredictability.PaperInterface.paper_educational_option_dia_variability_six_statement
```

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 48

Source locator: cited publication, lines 998-1003

Verbatim source input

```
\begin{restatable}[Additive Variability Bound]{theorem}{addvar}
\label{thm:additive-variability}
If  $\$choice\$$  is a sequential composition of  $\$q\$$ -acceptant,  $\$l\$$ -unstable functions with variabilities  $\$m_1, \dots, m_n\$$ , then
 $\hookrightarrow \$choice\$$  has variability at most  $\sum_{i=1}^n m_i$ .
\end{restatable}
```

Note that this is only an upper bound; we could trivially construct cases where this is not tight. For example, compose a
 $\hookrightarrow \$choice\$$ of two $\$q\$$ -representative choice functions with the same underlying ordering. On the other hand, it is also easy to
 $\hookrightarrow$ construct instances in which this upper bound is tight. Consider the composition of $\$m\$$ queues which each rank a student by a
 $\hookrightarrow$ different subject area exam. Then the addition of a new student to the applicant pool could displace any of $\$m\$$ students,
 $\hookrightarrow$ depending on the subject area.

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {qs ms : List ℕ}
{Cs : List (DGD26AdmissionsPredictability.PaperChoiceRule α)},
List.Forall₂
  (fun q C =>
    DGD26AdmissionsPredictability.paper_choice_function_feasible C ∧
    DGD26AdmissionsPredictability.paper_definition_q_acceptance q C ∧
    DGD26AdmissionsPredictability.paper_definition_d_instability 1 C)
  qs Cs →
List.Forall₂ (fun m C => DGD26AdmissionsPredictability.paper_definition_variability_at_most m C) ms Cs →
DGD26AdmissionsPredictability.paper_definition_variability_at_most ms.sum
(DGD26AdmissionsPredictability.paper_definition_sequential_composition Cs)
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_sequential_additive_variability_bound_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 49

Source locator: cited publication, lines 850-879

Verbatim source input

```
\begin{restatable}{theorem}{appendremove}
\label{thm:append-remove-size}
For  $q$ -acceptant, 1-unstable  $\mathcal{X}$ , and  $|\mathcal{X}| \geq 2q$ , the maximum size of the waitlisted set is equal to the maximum
 $\hookrightarrow$  size of the borderline set; that is,
\begin{equation}
\max_{X \subseteq \mathcal{X}} |V \setminus \text{choice}(X)| = \max_{X \subseteq \mathcal{X}} |A \setminus \text{choice}(X)|
\end{equation}
\end{restatable}

In short, if there is a set  $X$  with a waitlisted set of size  $u$ , there must be a set  $X_0$  with a borderline set of size (at
 $\hookrightarrow$  least)  $u$ , and vice versa. %

\begin{proof}
To prove equality, we will prove that 1)  $\max_{X \subseteq \mathcal{X}} |V \setminus \text{choice}(X)| \geq \max_{X \subseteq \mathcal{X}} |A \setminus \text{choice}(X)|$ 
 $\hookrightarrow$   $A \setminus \text{choice}(X)$  and then that 2)  $\max_{X \subseteq \mathcal{X}} |V \setminus \text{choice}(X)| \leq \max_{X \subseteq \mathcal{X}} |A \setminus \text{choice}(X)|$ .

\textbf{Part 1).}
To prove the first statement, assume that there is some  $X$  where the waitlisted set is of cardinality at least  $u$ , so  $x^*_1, \dots, x^*_u \in A \setminus \text{choice}(X)$ .

Then (recall that  $\mathcal{X}$  is consistent) there are  $X_1, \dots, X_u \subseteq X$  where  $X_i = X \setminus \{x^*_i\}$  such that
 $\hookrightarrow$   $\text{choice}(X_i) = x^*_i \cup \text{choice}(X) \setminus \{x^*_i\}$  for  $i \in \{1, \dots, u\}$ . Then consider  $X_0 = \bigcap_{i=1}^u X_i = X \setminus \{x^*_1, \dots, x^*_u\}$ ,
 $\hookrightarrow$   $\text{choice}(X_0) = \{x^*_1, \dots, x^*_u\} \cup \text{choice}(X)$ . It must be that  $\text{choice}(X_0) = \{x^*_1, \dots, x^*_u\} \cup \text{choice}(X)$ 
 $\hookrightarrow$   $\setminus \{x^*_1, \dots, x^*_u\}$  by substitutability. I claim that  $\{x^*_1, \dots, x^*_u\} \subseteq V \setminus \text{choice}(X_0)$ , which
 $\hookrightarrow$  means  $|V \setminus \text{choice}(X_0)| \geq u$ , which would complete the proof.

Assume for contradiction that this is untrue;  $\{x^*_1, \dots, x^*_u\} \not\subseteq V \setminus \text{choice}(X_0)$ . Then some  $x^*_i$ , say
 $\hookrightarrow$   $x^*_u$  (without loss of generality), is not in  $V \setminus \text{choice}(X_0)$ , which means that  $x^*_u \in \text{choice}(X_0)$ .
Consider the makeup of  $\text{choice}(X_0)$ . Recall that  $\text{choice}(X_0) = \bigcap_{i=1}^u \text{choice}(X_i) = X \setminus \{x^*_1, \dots, x^*_u\}$ .
 $\hookrightarrow$   $\{x^*_1, \dots, x^*_u\}$ . Because of substitutability, as a subset of  $X$ , we have  $\text{choice}(X) \setminus \{x^*_1, \dots, x^*_u\} \subseteq \text{choice}(X_0)$ .
 $\hookrightarrow$   $\{x^*_1, \dots, x^*_u\} \subseteq \text{choice}(X_0)$ . From substitutability, as a subset of  $X$ , we have  $\text{choice}(X) \setminus \{x^*_1, \dots, x^*_u\} \subseteq \text{choice}(X_0)$ .
 $\hookrightarrow$   $\{x^*_1, \dots, x^*_u\} \subseteq \text{choice}(X_0)$ . However, consider the size of  $\text{choice}(X_0)$ .  $|\text{choice}(X_0)| = |\text{choice}(X) \setminus \{x^*_1, \dots, x^*_u\}| + |\{x^*_1, \dots, x^*_u\}|$ .
 $\hookrightarrow$   $|\text{choice}(X_0)| = |\text{choice}(X)| - u + u = |\text{choice}(X)|$ . We also know that  $|\text{choice}(X)| = u-1$ , and as no
 $\hookrightarrow$   $x^*_i$  is in  $\text{choice}(X)$  by definition, all of these elements are distinct. As  $x^*_u$  remains by assumption, this adds up
 $\hookrightarrow$  to  $|\text{choice}(X_0)| = q+1$ , which violates  $q$ -acceptance and completes the proof.

\textbf{Part 2).}
The proof for 2) is extremely similar to the former. Begin with  $X_0$  with  $V \setminus \text{choice}(X_0) = \{x^*_1, \dots, x^*_m\}$ , with  $x^*_1, \dots, x^*_m$ 
 $\hookrightarrow$   $\{x^*_1, \dots, x^*_m\}$  such that  $\text{choice}(X_1) = \{x^*_1\} \cup \text{choice}(X_0) \setminus \{x^*_1\}$ . Let  $X = \bigcup_{i=1}^m X_i$ . Note that  $X_i = X_0 \cup \{x^*_i\}$ .
 $\hookrightarrow$   $\{x^*_1, \dots, x^*_m\}$  and  $X_i = X_0 \cup \{x^*_i\}$ . Note that  $X_i = X_0 \cup \{x^*_i\}$ . Note that  $X_i = X_0 \cup \{x^*_i\}$ . Note that  $X_i = X_0 \cup \{x^*_i\}$ .
 $\hookrightarrow$   $X_m$ . Because of  $1$ -instability,  $|\text{choice}(X_0) \setminus \text{choice}(X)| \leq |X_0 \setminus \text{choice}(X)| = m$ . Because of substitutability, any  $x$ 
 $\hookrightarrow$   $\in \text{choice}(X)$  is either in  $\text{choice}(X_0)$  or not in  $X_0$  and therefore in  $\{x^*_1, \dots, x^*_m\}$ .

I claim  $\text{choice}(X) = \{x^*_1, \dots, x^*_m\} \cup \text{choice}(X_0) \setminus \{x^*_1, \dots, x^*_m\}$ . Firstly,  $x^*_i \notin \text{choice}(X)$ 
 $\hookrightarrow$  because  $x^*_i \notin \text{choice}(X_i)$  and  $X_i \subseteq X$ , so that would be a substitutability violation. As
 $\hookrightarrow$   $\text{choice}(X_0) \setminus \{x^*_1, \dots, x^*_m\}$  has  $q-m$  elements, the only possible elements to fill  $\text{choice}(X)$  must be the
 $\hookrightarrow$   $m$  elements of  $\{x^*_1, \dots, x^*_m\}$ .

The same argument holds for any  $X_i$ , but use  $X_m$  without loss of generality for ease of notation;  $X_j \subseteq X_m$  for  $m \geq j$ ,
 $\hookrightarrow$  so  $x^*_j \notin \text{choice}(X_m)$  by substitutability. With  $\text{choice}(X_0) \setminus \{x^*_1, \dots, x^*_m\}$  being the
 $\hookrightarrow$  only  $q-m+1$  elements of  $X_0$  that can be in  $\text{choice}(X_m)$ , the other  $m-1$  elements must be  $\{x^*_1, \dots, x^*_{m-1}\}$ .
 $\hookrightarrow$  Then consider the definition of  $A \setminus \text{choice}(X)$  and see that  $\text{choice}(X \setminus \{x^*_i\}) \setminus \text{choice}(X) = \{x^*_i\}$  for
 $\hookrightarrow$  any  $i$ , with each  $x^*_i$  being unique.
\end{proof}
```

Expanded PaperInterface specification

```
\forall \{ \alpha : \text{Type } u_1 \} [ \text{inst} : \text{DecidableEq } \alpha ] [ \text{inst}_1 : \text{Fintype } \alpha ] \{ m q : \mathbb{N} \}
\{ C : \text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha \},
\text{DGD26AdmissionsPredictability.paper\_choice\_function\_feasible } C \rightarrow
\text{DGD26AdmissionsPredictability.paper\_definition\_q\_acceptance } q C \rightarrow
\text{DGD26AdmissionsPredictability.paper\_definition\_d\_instability } 1 C \rightarrow
(\text{DGD26AdmissionsPredictability.paper\_definition\_general\_variability\_exactly } m C \leftrightarrow
\text{DGD26AdmissionsPredictability.paper\_definition\_variability\_exactly } m C)
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_append_remove_variability_exact_equivalence_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 50

Source locator: cited publication, lines 576-580

Verbatim source input

```
\begin{restatable}[Triangle Inequality]{theorem}{triangle}
\label{thm:triangle-ineq}
For any choice function  $\text{\textit{choice}}$  and sets  $X_1 \subseteq X_2 \subseteq X_3$ ,  $\text{\textit{r\_choice}}$  obeys the triangle inequality;
 $\text{\textit{r\_choice}}(X_1, X_3) \leq \text{\textit{r\_choice}}(X_1, X_2) + \text{\textit{r\_choice}}(X_2, X_3)$ 
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] (C : DGD26AdmissionsPredictability.PaperChoiceRule α) {X₁ X₂ X₃ : Finset α},
  X₁ ⊆ X₂ →
  X₂ ⊆ X₃ →
    DGD26AdmissionsPredictability.paper_definition_choice_distance C X₁ X₃ ≤
      DGD26AdmissionsPredictability.paper_definition_choice_distance C X₁ X₂ +
        DGD26AdmissionsPredictability.paper_definition_choice_distance C X₂ X₃
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_choice_distance_triangle_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 51

Source locator: cited publication, lines 777-794

Verbatim source input

```
\begin{restatable}[Even Instability is Inconsistent]{theorem}{inconsistentstable}
\label{thm:even-inconsistency}
For any  $q$ -acceptant  $\mathcal{X}$ ,  $\mathcal{X}$  is inconsistent if and only if there exist  $X, X' = X \cup \{x\}$  where  $r_{\mathcal{X}}(X, X')$  is positive and even.
\end{restatable}

\textit{Intuition}: for  $\mathcal{X}$  to have an even but nonzero choice distance  $r_{\mathcal{X}}$ , the indicator term in
 $\text{lem:unstable-calc}$  must be zero; the added  $x$  is not selected. But that means  $x$  is a rejected element that still
has an impact, which violates consistency.
\begin{proof}
\textit{Informal}
First, we prove that an even and positive value of  $r_{\mathcal{X}}$  implies inconsistency. For  $\mathcal{X}$  to have an even but nonzero
choice distance  $r_{\mathcal{X}}$ , the indicator term in  $\text{lem:unstable-calc}$  must be zero; the added  $x$  is not selected.
But that means  $x$  affects selection without actually being chosen, which violates consistency.

Next, prove that consistency limits the values of  $r_{\mathcal{X}}$ . If  $\mathcal{X}$  is not consistent, then there is some  $X, X'$  where
the new element  $x$  is not chosen (so the indicator term in  $\text{lem:unstable-calc}$  is zero and  $r_{\mathcal{X}}$  is even), but
changes the chosen elements. This makes  $r_{\mathcal{X}}$  is positive under a fixed capacity constraint.

\textit{Formal}
Consider  $\text{lem:unstable-calc}$ . See that for  $r_{\mathcal{X}} \neq 0$ ,  $r_{\mathcal{X}}$  is odd if and only if  $x' \in \mathcal{X}(X')$  and
even if and only if  $x' \notin \mathcal{X}(X')$ . If  $x' \notin \mathcal{X}(X')$ , then  $\mathcal{X}(X') \subseteq X$ . %
So if  $x' \notin \mathcal{X}(X')$  but  $n > 0$ ,  $\mathcal{X}(X') \subseteq X$  but  $\mathcal{X}(X') \neq \mathcal{X}(X)$ , which proves
inconsistency.
Similarly, if  $\mathcal{X}$  is inconsistent, there is  $\mathcal{X}(X') \subseteq X$  but  $\mathcal{X}(X') \neq \mathcal{X}(X)$ . %
As  $\mathcal{X}(X') \neq \mathcal{X}(X)$ ,  $n > 0$ , but for  $\mathcal{X}(X') \subseteq X$  to be true,  $x' \notin \mathcal{X}(X')$ , and so
 $r_{\mathcal{X}}(X, X')$  is even.
\end{proof}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
  (¬DGD26AdmissionsPredictability.paper_definition_consistency C ↔
    ∃ X,
      ∃ x ∉ X,
        0 < DGD26AdmissionsPredictability.paper_definition_choice_distance C X (insert x X) ∧
        ∃ k, DGD26AdmissionsPredictability.paper_definition_choice_distance C X (insert x X) = 2 * k)
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_even_distance_iff_inconsistent_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 52

Source locator: cited publication, lines 683-707

Verbatim source input

```

\begin{restatable}{theorem}{Independent Rules are Substitutable and Monotonic}
\label{thm:independent-sub-mon}
 $\$choice\$$  is independent if and only if it is both substitutable and monotonic.
\end{restatable}

\textit{Intuition}: The only way for adding applicants to neither hurt those already accepted or help those already rejected is
 $\hookrightarrow$  to not affect them at all.

\begin{proof}
  Assume  $\$choice\$$  is independent.

  \textbf{Informal}
  Consider how independence implies substitutability and monotonicity. Independence means any element is always accepted or
   $\hookrightarrow$  always rejected regardless of applicant pool. Then any  $x$  that is accepted in  $X_1$  is still accepted in any subset of
   $\hookrightarrow$   $X_1$ , which satisfies the definition of substitutability, and is accepted in any superset of  $X_1$ , which satisfies
   $\hookrightarrow$  monotonicity.

  To prove that substitutability and monotonicity combine to form independence, consider the behavior they cover. Recall the
   $\hookrightarrow$  intuition behind substitutability --- removing applicants does not lead to rejecting those accepted, and the intuition
   $\hookrightarrow$  behind monotonicity --- adding applicants does not lead to rejecting those accepted. Given both properties, nothing can
   $\hookrightarrow$  affect an acceptance decision at all.

  \textbf{Formal}
  First, we assume independence and prove  $\$choice\$$ -stability.
  Recall \Cref{lem:substitutability-term}:  $\$choice\$$  is substitutable if and only if  $|X_1 \cap \text{choice}(X_2)| \leq |\text{choice}(X_1) \cap \text{choice}(X_2)|$  for all  $X_1 \subseteq X_2$ . Then for any  $x \in X_1$ , either  $x \in \text{choice}(X_1) \cap \text{choice}(X_2)$  or  $x \notin \text{choice}(X_1) \cap \text{choice}(X_2)$ , so  $|X_1 \cap \text{choice}(X_2)| = |\text{choice}(X_1) \cap \text{choice}(X_2)|$ . Then  $|X_1 \cap \text{choice}(X_2)| \leq |\text{choice}(X_1)|$  and so  $\$choice\$$  is substitutable. Similarly, recall \Cref{lem:monotonicity-term}:  $\$choice\$$  is monotonic if and only if  $|\text{choice}(X_1) \cap \text{choice}(X_2)| = 0$  for all  $X_1 \subseteq X_2$ . Any element  $x \in \text{choice}(X_1)$  is in  $X_2$  because  $\text{choice}(X_1) \subseteq X_1 \subseteq X_2$ . By definition of independence,  $x \in \text{choice}(X_2)$  for all  $X$  where  $x \in X$ , so if  $x \in \text{choice}(X_1)$ ,  $x \in \text{choice}(X_2)$ , so  $|\text{choice}(X_1) \cap \text{choice}(X_2)| = |\text{choice}(X_1)|$ .

  To prove that substitutability and monotonicity together imply independence, consider any  $x \in X$ . If  $x \in \text{choice}(X)$ , we can show that  $x \in \text{choice}(X')$  for any  $X'$  where  $x \in X'$ . By monotonicity,  $x \in \text{choice}(X' \cup X)$ , and so by substitutability,  $x \in \text{choice}(X')$ . If  $x \notin \text{choice}(X)$ , we can prove  $x \notin \text{choice}(X')$  for any  $X'$ . If we assumed for contradiction that  $x \in \text{choice}(X')$ , then the same argument as above proves  $x \in \text{choice}(X)$ , leading to contradiction.
\end{proof}

\begin{restatable}{corollary}{Only Independent Rules are 0-unstable}
\label{thm:independent-zero-unstable}
 $\$choice\$$  is  $\$choice\$$ -unstable if and only if it is independent, and therefore no  $\$q\$-acceptant  $\$choice\$$  can be  $\$choice\$$ -unstable.
\end{restatable}$ 
```

Expanded PaperInterface specification

$$\begin{aligned} & \forall \{ \alpha : \text{Type } u_1 \} [\text{inst} : \text{DecidableEq } \alpha] (C : \text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha), \\ & \quad \text{DGD26AdmissionsPredictability.paper_choice_function_feasible } C \rightarrow \\ & \quad (\text{DGD26AdmissionsPredictability.paper_definition_independence } C \leftrightarrow \\ & \quad \quad \text{DGD26AdmissionsPredictability.paper_definition_substitutability } C \wedge \\ & \quad \quad \text{DGD26AdmissionsPredictability.paper_definition_monotonicity } C) \end{aligned}$$

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_independent substitutable monotonic statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

Review row 53

Source locator: cited publication, lines 280-289

Verbatim source input

```
\begin{restatable}{theorem}{thminstability}
\label{thm:instability}
A  $q$ -acceptant choice function
\begin{enumerate}
\item cannot be  $q$ -unstable,
\item is exactly  $q$ -unstable if and only if it is substitutable,
\item can be tightly  $q$ -unstable for every  $1 \leq d \leq 2q$ .
\end{enumerate}
\end{restatable}%
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
  0 < q → ∀ {U : Finset α}, q < U.card → ¬DGD26AdmissionsPredictability.paper_definition_zero_instability C
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_no_zero_instability_under_capacity_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 54

Source locator: cited publication, lines 280-289

Verbatim source input

```
\begin{restatable}{theorem}{thminstability}
\label{thm:instability}
A  $q$ -acceptant choice function
\begin{enumerate}
\item cannot be  $0$ -unstable,
\item is exactly  $1$ -unstable if and only if it is substitutable,
\item can be tightly  $d$ -unstable for every  $1 \leq d \leq 2q$ .
\end{enumerate}
\end{restatable}%
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
  DGD26AdmissionsPredictability.paper_choice_function_feasible C →
  DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
  (DGD26AdmissionsPredictability.paper_definition_substitutability C ↔
   DGD26AdmissionsPredictability.paper_definition_d_instability 1 C)
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_substitutability_one_instability_equivalence_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 55

Source locator: cited publication, lines 280-289

Verbatim source input

```
\begin{restatable}{theorem}{thminstability}
\label{thm:instability}
A  $\$$ -acceptant choice function
\begin{enumerate}
\item cannot be  $\$$ -unstable,
\item is exactly  $\$$ -unstable if and only if it is substitutable,
\item can be tightly  $\$$ -unstable for every  $1 \leq d \leq 2q$ .
\end{enumerate}
\end{restatable}%

[Next verbatim source excerpt]

\subsection{Proof of \Cref{thm:instability}} \label{sec:instability-proof}

Recall \Cref{thm:instability}:
\thminstability*

Putting things together, the first statement of \Cref{thm:instability} comes from \Cref{thm:independent-zero-unstable} and the
 $\hookrightarrow$  second from \Cref{thm:sub-unstable-equiv}. The third
has roots in \Cref{lem:unstable-calc}, which allows us to calculate instability.

Rigorously, we can generate a  $\$$ -unstable  $\$$ -choice for any  $1 < d \leq 2q$ . Take any  $\$$ -unstable,  $\$$ -acceptant  $\$$ -choice
 $\hookrightarrow$  induced by a total order  $\$$ -succ (i.e, is a ranking). Let  $n$  be  $\frac{d}{2}$ , rounded down. Then let  $X^* = \{x^*_1, \dots,$ 
 $\hookrightarrow x^*_n\}$  be the  $n$   $\$$ -minimal elements of  $\$$ -succ (or any  $n$  elements that are not among the  $n$  maximal elements, if
 $\hookrightarrow$  there exist any.) Then modify  $\$$ -choice such that if  $X^* \subseteq X$  for any input  $X$ ,  $X^* \subseteq \text{choice}(X)$ . Then
 $\hookrightarrow$   $\$$ -choice is tightly  $2n-1$ -unstable. To make  $\$$ -choice  $2n$ -unstable, let there be some  $x'$  such that if  $X^* \subseteq X$ 
 $\hookrightarrow$  but  $x' \in X$  as well, then  $\$$ -no element of  $X^*$  is in  $\text{choice}(X)$ . %

The intuition for the construction is that  $X^*$  forms a highly desirable team, even though each element is not individually
 $\hookrightarrow$  desirable, nor are partial subsets of  $X^*$ .
In the inconsistent case (even instability)  $x'$  represents a candidate whose mere presence negates the value of  $X^*$  as a team,
 $\hookrightarrow$  even when not accepted.
```

Expanded PaperInterface specification

```
∀ (q d : ℕ),
  1 ≤ d  $\rightarrow$ 
  d ≤ 2 * q  $\rightarrow$ 
  ∃ β inst C,
    DGD26AdmissionsPredictability.paper_choice_function_feasible C  $\wedge$ 
    DGD26AdmissionsPredictability.paper_definition_q_acceptance q C  $\wedge$ 
    DGD26AdmissionsPredictability.paper_definition_tight_d_instability d C
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_theorem1_tight_all_d_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 56

Source locator: cited publication, lines 1097-1109

Verbatim source input

```
\begin{restatable}[Linear Assignment Instability]{theorem}{LAPstable}
\label{thm:lap-unstable}
Linear assignment problems are 1-unstable.
\end{restatable}

\textit{Intuition}: The ranking-like structure of linear assignment slots maintains substitutability, because each slot maintains
↪ a preference ordering over elements that is independent of the cohort.

\begin{proof}
Recall that for  $q$ -acceptant functions, substitutability and 1-instability are equivalent by \Cref{thm:sub-unstable-equiv}.

The proof is by contradiction; assume  $\text{choice}$  is not substitutable. As per \Cref{lem:unsubstitutability}, there exists
↪  $\text{choice}(X), \text{choice}(X')$  where some  $x^* \in X$  is not in  $\text{choice}(X)$  but  $x^* \in \text{choice}(X')$ .
By \Cref{lem:lap-ordering},  $\text{choice}(X) = \{x_{s_1}, \dots, x_{s_q}\}$  tells us we have  $x_{s_i} \succ_{s_i} x^*$  for all  $i \in \{1, \dots, q\}$ . For  $x^*$  to be in  $\text{choice}(X')$ ,  $x_{s_i}$  must also be in  $\text{choice}(X')$ , meaning that  $\text{choice}(X) \subseteq \text{choice}(X')$ . However, as  $x^* \in \text{choice}(X')$ ,  $|\text{choice}(X')| > |\text{choice}(X)|$ , which violates  $q$ -acceptance, leading to
↪ contradiction.
\end{proof}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {σ : Type u_2} [inst_1 : DecidableEq σ] [inst_2 : Fintype σ] {w : α → σ → ℝ}
(hwell : DGD26AdmissionsPredictability.LAP.Assignment.WellPosedObjective w),
DGD26AdmissionsPredictability.paper_definition_d_instability 1
(DGD26AdmissionsPredictability.paperLAPChoiceRule w hwell)
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_lap_assignment_one_instability_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 57

Source locator: cited publication, lines 1111-1129

Verbatim source input

```
\begin{restatable}[Linear Assignment Variability]{theorem}{LAPvariability}
\label{thm:lap-variability}
The variability of a linear assignment problem is upper bounded by the number of distinct preference orderings induced by its
  ↪ slots.
\end{restatable}

\textit{Intuition}:
The proof follows from the observation that, if slot  $s$  ranks some other element  $x$  in the chosen set higher than the element
  ↪  $x$   $s$  currently assigned to  $s$ ,  $x$  cannot be in the variable set. Then, if  $k$  slots (say, slots  $s_1, \dots, s_k$ , with
  ↪ assigned elements  $x_{s_1}, \dots, x_{s_k}$ ) induce the same preference ordering over elements, then only the minimal element
  ↪ in  $\{x_{s_1}, \dots, x_{s_k}\}$  according to that preference ordering can be in  $V_{\text{choice}(X)}$ .

\begin{proof}
We want to show that variability is upper bounded by the number of unique orderings. Recall that the variable set  $V_{\text{choice}(X)}$ 
  ↪  $\subseteq \text{choice}(X)$  is at most cardinality  $q$ .

I claim that for any  $X$  and  $\text{choice}(X)$ ,  $x_{s_i}$  is not in  $V_{\text{choice}(X)}$  if  $x_{s_i} \succ_{s_j} x_{s_j}$  for any  $j \neq i$ .
To be precise, if  $x_{s_i} \in \text{choice}(X)$ , and  $x_{s_i} \succ_{s_j} x_{s_j}$  for some  $j \neq i$ , then  $x_{s_i} \in \text{choice}(X')$ 
  ↪ for any  $X' = X \cup \{x\}$ .

The proof of this claim is by contradiction: assume there exists  $\text{choice}(X)$ ,  $\text{choice}(X')$  where some  $x_{s_i} \succ_{s_j}$ 
  ↪  $x_{s_j}$  but  $x_{s_i} \notin \text{choice}(X')$ . Firstly, because of the 1-instability proven in \Cref{thm:lap-unstable},
  ↪  $|\text{choice}(X) \setminus \text{choice}(X')| = 1$ , which means by assumption,  $\text{choice}(X) \setminus \text{choice}(X') = \{x_{s_i}\}$ , which
  ↪ further implies  $x_{s_j} \in \text{choice}(X')$ . However, \Cref{lem:lap-ordering} then applies to  $x_{s_j}$ , where  $x_{s_i}$ 
  ↪  $\succ_{s_j} x_{s_j}$  implies  $x_{s_i} \in \text{choice}(X')$ , leading to contradiction.

Secondly, note that if  $s_i, s_j$  induce the same total ordering, then either  $x_{s_i} \succ_{s_j} x_{s_j}$  or  $x_{s_j}$ 
  ↪  $\succ_{s_i} x_{s_i}$  (and the same for  $\text{choice}(X)$ ) for any  $\text{choice}(X)$ , as a property of any total ordering. Then one of
  ↪ either  $x_{s_j}$  or  $x_{s_i}$  are not in  $V_{\text{choice}(X)}$ . It is intuitive to see that this generalizes: if  $s_1, \dots, s_j$  all
  ↪ induce the same ordering  $\succ_{s_j}$ , there is exactly one minimal element in  $x_{s_1}, \dots, x_{s_j}$ , because any finite
  ↪ subset of a totally ordered set is well-ordered.
Say that  $x_{s_j}$  is the minimal element, w.l.o.g.; then none of  $x_{s_1}, \dots, x_{s_{j-1}}$  are in  $V_{\text{choice}(X)}$ . More
  ↪ generally, let there be  $m$  unique orderings, which we notate  $\succ_1, \dots, \succ_m$ . Let  $k \in \{1, \dots, m\}$  and let there
  ↪ be  $c_k$  slots that induce  $\succ_k$ . Then we have
defined sets of size  $c_k - 1$  elements that cannot be in  $V_{\text{choice}(X)}$ . Then  $|V_{\text{choice}(X)}| \leq m$ , and therefore the overall
  ↪ variability is at most  $m$ .
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {σ : Type u_2} [inst_2 : DecidableEq σ]
  [inst_3 : Fintype σ] {w : α → σ → ℝ} (hw : DGD26AdmissionsPredictability.LAP.Assignment.WellPosedObjective w),
  (∀ (s : σ), DGD26AdmissionsPredictability.LAP.Assignment.SlotNoTies w s) →
  DGD26AdmissionsPredictability.paper_definition_variability_at_most
    (DGD26AdmissionsPredictability.LAP.Assignment.distinctSlotOrderCount w)
    (DGD26AdmissionsPredictability.paperLAPChoiceRule w hw)
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_lap_assignment_slot_order_class_variability_of_unique_global_optima_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 58

Source locator: cited publication, lines 910-923

Verbatim source input

```
\begin{restatable}[ $q$ -Representativeness and  $l$ -Variability are Equivalent]{theorem}{rankingvariability}
\label{thm:ranking-variability}
 $\text{\choice}$  is  $q$ -representative if and only if it is  $q$ -acceptant and  $l$ -unstable with variability  $l$ .
\end{restatable}
\begin{proof}
We prove this in two parts: first, that  $q$ -representativeness implies  $l$ -variability, and then the converse.

\paragraph{Direction 1} %
Note that  $q$ -representativeness implies  $q$ -acceptance by definition.

Next, we prove that  $q$ -representativeness implies  $l$ -instability. Assume for contradiction that some  $q$ -representative
 $\hookrightarrow$   $\text{\choice}$  is not substitutable (and therefore not  $l$ -unstable). Then there exists (by \Cref{lem:unsubstitutable}) some
 $\hookrightarrow$   $x^*$ ,  $x'$ ,  $X$  and  $X' = X \cup \{x'\}$  where  $x^* \in X$  and  $x^* \in \text{\choice}(X)$  but  $x^* \notin \text{\choice}(X)$ . By
 $\hookrightarrow$   $q$ -acceptance, there must be at least one  $x$  in  $\text{\choice}(X)$  that is not in  $\text{\choice}(X')$  (which must take the place of
 $\hookrightarrow$   $x^*$ ). However,  $\text{\choice}(X')$  implies that  $x^* \succ x$ , while  $\text{\choice}(X)$  implies that  $x \succ x^*$  --- violating
 $\hookrightarrow$  asymmetry and leading us to contradiction.

Lastly, proving that  $q$ -representative  $\text{\choice}$  leads to  $m=l$  is relatively intuitive. For any  $X$ ,  $\text{\choice}(X)$  are the  $q$ 
 $\hookrightarrow$  highest-ranked elements of  $X$ . Denote  $X = \{x_1, x_2, \dots, x_k\}$  where  $|X| = k$  and  $x_1 \succ x_2 \succ \dots \succ x_k$ .
 $\hookrightarrow$  Then  $\text{\choice}(X) = \{x_1, \dots, x_{q^*}\}$  where  $q^* = \min\{q, k\}$ .
Assume for contradiction that there is a  $q$ -representative  $\text{\choice}$  with variability  $m > l$ . Then there exists some  $X$  where
 $\hookrightarrow$   $|V_{\text{\choice}(X)}| > l$ ; say that  $\{x^*_a, x^*_b\} \subseteq V_{\text{\choice}(X)}$ . Then there exists some  $x'_a$  where  $\text{\choice}(X) \setminus \{x^*_a\} \cup x'_a = \{x^*_a\}$  and similar  $x'_b$  for  $x^*_b$ . The first implies  $x^*_b$  is ranked above
 $\hookrightarrow$   $x^*_a$ , but the second implies the opposite. Then choices cannot be made according to a static ranking and  $\text{\choice}$  is not
 $\hookrightarrow$   $q$ -representative, completing the proof by contradiction. %
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {q : ℕ}
{C : DGD26AdmissionsPredictability.PaperChoiceRule α},
DGD26AdmissionsPredictability.paper_choice_function_feasible C →
DGD26AdmissionsPredictability.paper_definition_q_representativeness q C →
0 < q →
q < Fintype.card α →
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C ∧
DGD26AdmissionsPredictability.paper_definition_d_instability 1 C ∧
DGD26AdmissionsPredictability.paper_definition_variability_exactly 1 C
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_q_representative_forward_exact_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 59

Source locator: cited publication, lines 910-950

Verbatim source input

```
\begin{restatable}[ $q$ -Representativeness and  $l$ -Variability are Equivalent]{theorem}{rankingvariability}
\label{thm:ranking-variability}
 $\text{\choice}$  is  $q$ -representative if and only if it is  $q$ -acceptant and  $l$ -unstable with variability  $l$ .
\end{restatable}
\begin{proof}
We prove this in two parts: first, that  $q$ -representativeness implies  $l$ -variability, and then the converse.

\paragraph{Direction 1} %
Note that  $q$ -representativeness implies  $q$ -acceptance by definition.

Next, we prove that  $q$ -representativeness implies  $l$ -instability. Assume for contradiction that some  $q$ -representative
 $\rightarrow$   $\text{\choice}$  is not substitutable (and therefore not  $l$ -unstable). Then there exists (by \Cref{lem:unsubstitutable}) some
 $\rightarrow$   $x^* \in X$ ,  $x \in X$ , and  $x' \in X$  such that  $x^* \in \text{\choice}(X)$  and  $x^* \in \text{\choice}(X')$  but  $x^* \notin \text{\choice}(X)$ . By
 $\rightarrow$   $q$ -acceptance, there must be at least one  $x \in \text{\choice}(X)$  that is not in  $\text{\choice}(X')$  (which must take the place of
 $\rightarrow$   $x^*$ ). However,  $\text{\choice}(X')$  implies that  $x^* \succ x$ , while  $\text{\choice}(X)$  implies that  $x \succ x^*$  --- violating
 $\rightarrow$  asymmetry and leading us to contradiction.

Lastly, proving that  $q$ -representative  $\text{\choice}$  leads to  $m=1$  is relatively intuitive. For any  $X$ ,  $\text{\choice}(X)$  are the  $q$ 
 $\rightarrow$  highest-ranked elements of  $X$ . Denote  $X = \{x_1, x_2, \dots, x_k\}$  where  $|X| = k$  and  $x_1 \succ x_2 \succ \dots \succ x_k$ .
 $\rightarrow$  Then  $\text{\choice}(X) = \{x_1, \dots, x_{q^*}\}$  where  $q^* = \min\{q, k\}$ .
Assume for contradiction that there is a  $q$ -representative  $\text{\choice}$  with variability  $m > 1$ . Then there exists some  $X$  where
 $\rightarrow$   $|V_{\text{\choice}(X)}| > 1$ ; say that  $\{x^* a, x^* b\} \subseteq V_{\text{\choice}(X)}$ . Then there exists some  $x' a$  where  $\text{\choice}(X)$ 
 $\rightarrow$   $\setminus \text{\choice}(X \cup x' a) = \{x^* a\}$  and similar  $x' b$  for  $x^* b$ . The first implies  $x^* b$  is ranked above
 $\rightarrow$   $x^* a$ , but the second implies the opposite. Then choices cannot be made according to a static ranking and  $\text{\choice}$  is not
 $\rightarrow$   $q$ -representative, completing the proof by contradiction. %

\paragraph{Direction 2}
Assume  $q$ -acceptance, variability 1, and instability 1. Now we prove  $\text{\choice}$  is  $q$ -representative. To be precise, I take this
 $\rightarrow$  to mean that  $\text{\choice}$  is indistinguishable from some  $q$ -representative  $\text{\choice}$ ;  $\text{\choice}$  could be represented by some
 $\rightarrow$  total ordering over elements of  $X$ . We show this strict total ordering constructively.

Define  $\text{\succ}$  such that, if  $x_a \in \text{\choice}(X)$  but  $x_b \notin \text{\choice}(X)$  for any  $x_a, x_b \in X$ ,  $x_a \succ x_b$ . Note
 $\rightarrow$  that this implies that the existence of some set  $A$  where  $x_a \in \text{\choice}(A)$  and  $A \setminus \text{\choice}(A) = \{x_b\}$  by consistency
 $\rightarrow$  (e.g.  $A = \text{\choice}(X) \cup \{x_b\}$ ). This also implies  $A \setminus \text{\choice}(A) = \{x_b\}$  where  $x_a, x_b \in \text{\choice}(A)$ ,  $V_{\text{\choice}(A)} = \{x_b\}$ ,
 $\rightarrow$  where  $A = A \setminus \text{\choice}(A) \cup \{x_b\}$  for any  $x \neq x_a, x_b$ .

Then, we need to prove this relation is asymmetric, transitive, and completeness (or rather, is indistinguishable from a complete
 $\rightarrow$  relation when examining the induced choice function).

\paragraph{Asymmetry}
First, we show asymmetry. Assume for contradiction that  $a \succ b$  and  $b \succ a$ . Then there exists  $X_a$  where  $a, b \in X_a$ 
 $\rightarrow$   $X_a \setminus \text{\choice}(X_a) = \{a\}$  and  $b \notin \text{\choice}(X_a)$ , and a similar  $X_b$  where  $a \notin \text{\choice}(X_b)$  and so on.

Let  $X_a = \text{\choice}(X_a) \cup \{b\}$  (we can construct such out of any  $X_a$ , by consistency), where  $A \setminus \text{\choice}(X_a) = \{b\}$ , and
 $\rightarrow$  denote  $\text{\choice}(X_a) = \{x^a_1, \dots, x^a_{q-1}\}$ . Construct  $X_b$  similarly. Further, let  $X'_a = X_a \setminus \text{\choice}(X_a)$ 
 $\rightarrow$   $\{x^a_1\}$ , and note that  $V_{\text{\choice}(X'_a)} = \{b\}$ .

Then from here, we add the elements of  $X_b$  to  $X'_a$ , constructing  $X_1, X_2$ , and so forth.

Let  $X_0 = X'_a$  and sequentially construct a series of  $X_t$ . Add  $x^b_t$  to  $X_{t-1}$ . If  $x_b \notin \text{\choice}(X_{t-1}) \cup$ 
 $\rightarrow$   $\{x^b_t\}$ , then let  $X_t = X_{t-1} \cup \{x^b_t\}$ , and  $V_{\text{\choice}(X_t)} = \{b\}$  by \Cref{lem:consistent-remove}. If  $x^b_t \in$ 
 $\rightarrow$   $\text{\choice}(X_{t-1}) \cup \{x^b_t\}$ , then  $b \notin \text{\choice}(X_{t-1}) \cup \{x^b_t\}$  and  $A \setminus \text{\choice}(X_{t-1}) \cup \{x^b_t\}$ 
 $\rightarrow$   $= \{b\}$  (by \Cref{lem:a-v-equal}). Then let  $X_t = \{x^b_t\} \cup X'_a \setminus \text{\choice}(X'_a)$  where  $x^a_{i-1} \notin \text{\choice}(X_{t-1})$ 
 $\rightarrow$  but  $x^a_i \in \text{\choice}(X_{t-1})$ , and  $V_{\text{\choice}(X_t)} = \{b\}$  (again, by \Cref{lem:a-v-equal}). Note that  $a \in \text{\choice}(X_t)$ ,  $b \in$ 
 $\rightarrow$   $\text{\choice}(X_t)$ , and specifically  $b \in V_{\text{\choice}(X_t)}$ , for all  $t \in \{0, \dots, q-2\}$ .

Then  $X_{q-2}$  will contain  $x^b_1, \dots, x^b_{q-2}$ . Let  $X^* = X_{q-2} \cup \{x^b_{q-1}\}$ . Note that adding  $x^b_{q-1}$  to
 $\rightarrow$   $X_{q-2}$  makes  $X^* \supseteq X_b$ . Then, if  $a \in \text{\choice}(X^*)$ , substitutability is violated, as  $a \notin \text{\choice}(X_b)$ .
 $\rightarrow$   $\text{\choice}(X_b)$ . If  $x^b_{q-1} \notin \text{\choice}(X^*)$ , then  $\text{\choice}(X_{q-2}) = \text{\choice}(X^*)$  by consistency, and  $a \in$ 
 $\rightarrow$   $\text{\choice}(X_{q-2})$ . If instead  $x^b_{q-1} \in \text{\choice}(X^*)$ , recall that  $V_{\text{\choice}(X_{q-2})} = \{b\}$ , so  $a \notin \text{\choice}(X_{q-2})$ ,
 $\rightarrow$   $V_{\text{\choice}(X_{q-2})} = \{b\}$ , and  $a$  remains in  $\text{\choice}(X^*)$ . Therefore, either possibility leads to contradiction, proving that
 $\rightarrow$   $\text{\succ}$  is asymmetric.

\paragraph{Transitivity}
Say that  $a \succ b$  and  $b \succ c$  and we want to prove  $a \succ c$ . If  $a \succ b$ , there exists some  $X$  where  $a, b \in X$ ,
 $\rightarrow$   $a \in \text{\choice}(X)$ , and  $b \notin \text{\choice}(X)$ . Let  $X$  be  $\text{\choice}(X) \cup \{b\}$ , where  $a \in \text{\choice}(X)$ ,  $b \notin$ 
 $\rightarrow$   $\text{\choice}(X)$  by consistency. Then consider  $X \cup \{c\}$ . By asymmetry,  $c \notin \text{\choice}(X)$ , so  $c \notin \text{\choice}(X \cup$ 
 $\rightarrow$   $\{c\})$ . By consistency, then,  $\text{\choice}(X \cup \{c\}) = \text{\choice}(X)$ , so  $a \in \text{\choice}(X \cup \{c\})$  and  $a \succ c$ ,
 $\rightarrow$  completing the proof.

\paragraph{(Almost)-Completeness}
See that this relation must exist for all but  $q$  elements --- and any arbitrary linear extension thereof generates the same
 $\rightarrow$  choice function behavior.

For any  $a, b$ , let there be some  $X_0$  where  $a, b \notin \text{\choice}(X_0)$ . Then remove elements of  $\text{\choice}(X_t)$  in a sequence
 $\rightarrow$   $X_0, \dots, X_t$  until  $A \setminus \text{\choice}(X_t) = \{a\}$  or  $\{b\}$ . This must happen for some value of  $t$  because when  $|X_t| = q+1$ ,
 $\rightarrow$  at least one of  $a$  or  $b$  are in  $\text{\choice}(X_t)$ , and due to  $l$ -instability, both  $a$  and  $b$  cannot be added to
 $\rightarrow$   $\text{\choice}(X_t)$  at once. The only exception is if there is no such  $X_0$  where  $a, b \notin \text{\choice}(X_0)$ , which can only be
 $\rightarrow$  true if  $a \in \text{\choice}(X)$  for all  $X$  (and similarly for  $b$ ). (If  $a \notin \text{\choice}(X)$  for some  $X$ ,
 $\rightarrow$  then substitutability implies  $a \notin \text{\choice}(X \cup \{b\})$ . However, there can only be  $q$  such elements (by
 $\rightarrow$   $q$ -acceptance). Note that any linear extension of this partial order will produce the same choice function behavior; that
 $\rightarrow$  is, any constructed total ordering that otherwise aligns with  $\text{\succ}$  will produce the same choice behavior, and thus,
 $\rightarrow$   $\text{\choice}$  can be represented by any such strict total ordering.
\end{proof}
```

Expanded PaperInterface specification

```

∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {q : ℕ}
{C : DGD26AdmissionsPredictability.PaperChoiceRule α},
DGD26AdmissionsPredictability.paper_choice_function_feasible C →
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
DGD26AdmissionsPredictability.paper_definition_d_instability 1 C →
DGD26AdmissionsPredictability.paper_definition_variability_exactly 1 C →
0 < q → q < Fintype.card α → DGD26AdmissionsPredictability.paper_definition_q_representativeness q C

```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_q_representative_converse_exact_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 60

Source locator: cited publication, lines 962-996

Verbatim source input

```

\paragraph{Sequentially Composed Substitutable Functions Are Substitutable}
Similarly, we show sequential compositions of substitutable functions are also substitutable.
\begin{restatable}[Composition Preserves Substitutability]{theorem}{seqsub}
\label{thm:composition-substitutable}
Sequential composition of substitutable functions yields a substitutable function.
\end{restatable}

\textit{Intuition}:
``Appending'' a substitutable choice function  $\text{\textbackslash choice\_2}$  before another substitutable function  $\text{\textbackslash choice\_1}$  affects the inputs to
 $\text{\textbackslash choice\_1}$  in a structured way.
\begin{proof}
Proof by induction.
In the base case, for  $n = 1$ ,  $\text{\textbackslash choice}(X) = \text{\textbackslash choice\_1}(X)$ , and as  $\text{\textbackslash choice\_1}$  is given to be substitutable,  $\text{\textbackslash choice}$  is as
well.

For the inductive step, show that  $\text{\textbackslash choice}^{n+1}$  is substitutable when comprised of  $\text{\textbackslash choice}_{n+1}$ ,  $\text{\textbackslash choice}_n$ , ...,
 $\text{\textbackslash choice\_1}$ , assuming that  $\text{\textbackslash choice}^n$  comprised of  $\text{\textbackslash choice}_n, \dots, \text{\textbackslash choice\_1}$  is substitutable.

The definition of substitutability is that  $X \text{\textbackslash subseteq} X'$  implies  $\text{\textbackslash choice}(X') \text{\textbackslash cap} X \text{\textbackslash subseteq} \text{\textbackslash choice}(X)$ . So we need to
show  $\text{\textbackslash choice}^{n+1}(X') \text{\textbackslash cap} X \text{\textbackslash subseteq} \text{\textbackslash choice}^{n+1}(X)$ .

First, note that  $\text{\textbackslash choice}^{n+1}(X) = \text{\textbackslash choice}_{n+1}(X) \text{\textbackslash cup} \text{\textbackslash choice}^n(X \text{\textbackslash setminus} \text{\textbackslash choice}_{n+1}(X))$ .
Let  $S = X \text{\textbackslash setminus} \text{\textbackslash choice}_{n+1}(X)$  and  $S' = X' \text{\textbackslash setminus} \text{\textbackslash choice}_{n+1}(X')$ . As  $S \text{\textbackslash subseteq} X'$ , we only need to show
that no elements of  $S$  are elements of  $\text{\textbackslash choice}_{n+1}(X')$ . As  $\text{\textbackslash choice}_{n+1}$  is substitutable,  $\text{\textbackslash choice}_{n+1}(X') \text{\textbackslash cap}$ 
 $X = \text{\textbackslash choice}_{n+1}(X)$ . So, any element of  $S$  that is in  $\text{\textbackslash choice}_{n+1}(X')$  must be in  $\text{\textbackslash choice}_{n+1}(X)$ , and
therefore cannot be in  $S = X \text{\textbackslash setminus} \text{\textbackslash choice}_{n+1}(X)$ . Therefore,  $S \text{\textbackslash subseteq} S'$ .

Now, we can apply the induction hypothesis, so we assume  $\text{\textbackslash choice}^n$  is substitutable to get

\begin{align*}
&\text{\textbackslash choice}^n(S') \text{\textbackslash cap} S \text{\textbackslash subseteq} \text{\textbackslash choice}^n(S) \text{ \&\& \textit{induction hypothesis}} \\
&\text{\textbackslash choice}^n(S') \text{\textbackslash cap} \text{\textbackslash left}(X \text{\textbackslash setminus} \text{\textbackslash choice}_{n+1}(X) \text{\textbackslash right}) \text{\textbackslash subseteq} \text{\textbackslash choice}^n(S) \text{ \&\& \textit{substitute out } S} \\
&\text{\textbackslash left}(\text{\textbackslash choice}^n(S') \text{\textbackslash cap} X \text{\textbackslash right}) \text{\textbackslash setminus} \text{\textbackslash left}(\text{\textbackslash choice}^n(S') \text{\textbackslash cap} \text{\textbackslash choice}_{n+1}(X) \text{\textbackslash right}) \text{\textbackslash subseteq} \\
&\text{\textbackslash choice}^n(S) \text{ \&\& \textit{intersection commutes}} \\
&\text{\textbackslash choice}_{n+1}(X) \text{\textbackslash cup} \text{\textbackslash left}(\text{\textbackslash left}(\text{\textbackslash choice}^n(S') \text{\textbackslash cap} X \text{\textbackslash right}) \text{\textbackslash setminus} \text{\textbackslash left}(\text{\textbackslash choice}^n(S') \text{\textbackslash cap} \\
&\text{\textbackslash choice}_{n+1}(X) \text{\textbackslash right}) \text{\textbackslash right}) \text{\textbackslash subseteq} \text{\textbackslash choice}_{n+1}(X) \text{\textbackslash cup} \text{\textbackslash choice}^n(S) \text{ \&\& \textit{add } \text{\textbackslash choice}_{n+1}(X)} \\
&\text{\textbackslash intertext}{\textit{Note that } \text{\textbackslash choice}_{n+1}(X) \text{\textbackslash supseteq} \text{\textbackslash left}(\text{\textbackslash choice}^n(S') \text{\textbackslash cap} \text{\textbackslash choice}_{n+1}(X) \text{\textbackslash right})}, \text{ so the set} \\
&\text{\textbackslash difference can be simplified out:} \\
&\text{\textbackslash choice}_{n+1}(X) \text{\textbackslash cup} \text{\textbackslash left}(\text{\textbackslash choice}^n(S') \text{\textbackslash cap} X \text{\textbackslash right}) \text{\textbackslash subseteq} \text{\textbackslash choice}_{n+1}(X) \text{\textbackslash cup} \text{\textbackslash choice}^n(S) \text{ \&\& } \\
&\text{\textit{equivalent}} \text{\textbackslash \&\& \textit{Note that for any sets } A, B, C, (A \text{\textbackslash cup} B) \text{\textbackslash cap} C = (A \text{\textbackslash cap} C) \text{\textbackslash cup} (B \text{\textbackslash cap} C)} \\
&\text{\textbackslash subseteq} A \text{\textbackslash cup} (B \text{\textbackslash cap} C), \text{ so:} \\
&\text{\textbackslash left}(\text{\textbackslash choice}_{n+1}(X) \text{\textbackslash cup} \text{\textbackslash choice}^n(S') \text{\textbackslash right}) \text{\textbackslash cap} X \text{\textbackslash subseteq} \text{\textbackslash choice}_{n+1}(X) \text{\textbackslash cup} \text{\textbackslash choice}^n(S) \text{ \&\& \textit{}} \\
&\text{\textbackslash left}(\text{\textbackslash choice}_{n+1}(X) \text{\textbackslash cup} \text{\textbackslash choice}^n(X' \text{\textbackslash setminus} \text{\textbackslash choice}_{n+1}(X')) \text{\textbackslash right}) \text{\textbackslash cap} X \text{\textbackslash subseteq} \text{\textbackslash choice}_{n+1}(X) \text{\textbackslash cup} \\
&\text{\textbackslash choice}^n(X \text{\textbackslash setminus} \text{\textbackslash choice}_{n+1}(X)) \text{ \&\& \textit{substitute out } S, S'} \\
&\text{\textbackslash choice}_{n+1}(X') \text{\textbackslash cap} X \text{\textbackslash subseteq} \text{\textbackslash choice}^{n+1}(X) \text{ \&\& \textit{definition of } \text{\textbackslash choice}^{n+1}}
\end{align*}

Which is the definition of substitutability, completing the proof.
\end{proof}

```

Expanded PaperInterface specification

$$\forall \{\alpha : \text{Type } u_1\} [\text{inst} : \text{DecidableEq } \alpha] \{\text{Cs} : \text{List } (\text{DGD26AdmissionsPredictability.PaperChoiceRule } \alpha)\},$$

$$(\forall C \in \text{Cs}, \text{DGD26AdmissionsPredictability.paper_choice_function_feasible } C) \rightarrow$$

$$(\forall C \in \text{Cs}, \text{DGD26AdmissionsPredictability.paper_definition_substitutability } C) \rightarrow$$

$$\text{DGD26AdmissionsPredictability.paper_definition_substitutability}$$

$$(\text{DGD26AdmissionsPredictability.paper_definition_sequential_composition } \text{Cs})$$

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper sequential composition substitutable statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 61

Source locator: cited publication, lines 733-745

Verbatim source input

```
\begin{restatable}{theorem}{BoundedDistance}
\label{thm:sub-accept-consistent}
Any  $q$ -acceptant, substitutable  $\text{\textit{\textit{choice}}}$  is consistent.
\end{restatable}

\begin{proof}
Consider  $X_1, X_2$  where  $\text{\textit{\textit{choice}}}(X_2) \subseteq X_1 \subseteq X_2$  for substitutable,  $q$ -acceptant  $\text{\textit{\textit{choice}}}$ . Substitutability
 $\rightarrow$  guarantees that if  $x \in \text{\textit{\textit{choice}}}(X_2)$  and  $x \in X_1$ ,  $x \in \text{\textit{\textit{choice}}}(X_1)$ . By construction, this applies to all  $x \in$ 
 $\rightarrow \text{\textit{\textit{choice}}}(X_2)$ , because  $\text{\textit{\textit{choice}}}(X_2) \subseteq X_1$ ; therefore,  $\text{\textit{\textit{choice}}}(X_2) \subseteq \text{\textit{\textit{choice}}}(X_1)$ . As  $|\text{\textit{\textit{choice}}}(X_2)| =$ 
 $\rightarrow |\text{\textit{\textit{choice}}}(X_1)|$ , and  $\text{\textit{\textit{choice}}}(X_1), \text{\textit{\textit{choice}}}(X_2)$  are finite sets, this means that  $\text{\textit{\textit{choice}}}(X_2) = \text{\textit{\textit{choice}}}(X_1)$ , which is the
 $\rightarrow$  definition of consistency.
\end{proof}

\subsection{\textit{d}-Instability} \label{sec:d-unstable-appendix}

\subsubsection{Proof of \Cref{lem:unstable-calc}} \label{sec:calc-proof}
```

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] {q : ℕ} {C : DGD26AdmissionsPredictability.PaperChoiceRule α},
  DGD26AdmissionsPredictability.paper_definition_q_acceptance q C →
  DGD26AdmissionsPredictability.paper_definition_substitutability C →
  DGD26AdmissionsPredictability.paper_definition_consistency C
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_q_acceptant_substitutable_consistent_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 62

Source locator: cited publication, lines 315-322

Verbatim source input

```
\begin{restatable}{theorem}{thmvariability}
\label{thm:variability}
Consider a  $q$ -acceptant, 1-unstable choice function  $\mathcal{C}$  which
can be represented as the sequential composition of  $n$  choice functions, each characterized by a total order.
Then  $\mathcal{C}$  has a variability  $m$ , where  $1 \leq m \leq n$ . Furthermore,  $\mathcal{C}$  has variability  $1$  if and only if
 $\hookrightarrow$  it can be characterized by a single total order,  $n=1$ .
\end{restatable}
In the appendix, we discuss when this upper bound is tight. We also extend this result to choice functions which are not
 $\hookrightarrow$  explicitly defined as a fixed sequential composition, such as linear assignment problems.
\Cref{thm:lap-variability} addresses the context-dependent sequential queues which result from such optimization based admissions.

[Next verbatim source excerpt]

\subsection{Proof of \Cref{thm:variability}} \label{sec:variability-proof}

Recall \Cref{thm:variability}:
\thmvariability*

We see that the second claim comes exactly from \Cref{thm:ranking-variability}, and that combining it with
 $\hookrightarrow$  \Cref{thm:additive-variability} directly gives us the first claim.
```

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

For a sequential composition of total-order q -representative queues with positive total capacity on a universe larger than that
 $\hookrightarrow$ capacity, there exists m with $1 \leq m \leq n$ such that the composition has exact variability m .

Archival source anchor: cited publication, lines 315-322

Recorded basis DGD source clarification: Theorem 2's range is read on the positive-capacity, nontrivial queue domain.

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {qs : List ℕ}
{Cs : List (DGD26AdmissionsPredictability.PaperChoiceRule α)},
List.Forall₂
  (fun q C =>
    DGD26AdmissionsPredictability.paper_choice_function_feasible C ∧
    DGD26AdmissionsPredictability.paper_definition_q_representativeness q C)
qs Cs →
0 < qs.sum →
qs.sum < Fintype.card α →
  ∃ m,
    1 ≤ m ∧
    m ≤ Cs.length ∧
    DGD26AdmissionsPredictability.paper_definition_variability_exactly m
    (DGD26AdmissionsPredictability.paper_definition_sequential_composition Cs)
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_sequential_q_representative_variability_range_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 63

Source locator: cited publication, lines 315-323

Verbatim source input

```
\begin{restatable}{theorem}{thmvariability}
\label{thm:variability}
Consider a  $q$ -acceptant, 1-unstable choice function  $\mathcal{C}$  which
can be represented as the sequential composition of  $n$  choice functions, each characterized by a total order.
Then  $\mathcal{C}$  has a variability  $m$ , where  $1 \leq m \leq n$ . Furthermore,  $\mathcal{C}$  has variability  $1$  if and only if
 $\hookrightarrow$  it can be characterized by a single total order,  $n=1$ .
\end{restatable}
In the appendix, we discuss when this upper bound is tight. We also extend this result to choice functions which are not
 $\hookrightarrow$  explicitly defined as a fixed sequential composition, such as linear assignment problems.
\Cref{thm:lap-variability} addresses the context-dependent sequential queues which result from such optimization based admissions.
Importantly, Theorem~\ref{thm:variability} means that a choice function can be faithfully represented by a machine learning model
 $\hookrightarrow$  if and \emph{only if} it is characterized by a total ordering over  $\mathcal{X}$ .
```

[Next verbatim source excerpt]

```
\subsection{Proof of \Cref{thm:variability}} \label{sec:variability-proof}
```

```
Recall \Cref{thm:variability}:
\thmvariability*
```

We see that the second claim comes exactly from \Cref{thm:ranking-variability}, and that combining it with
 $\hookrightarrow$ \Cref{thm:additive-variability} directly gives us the first claim.

Approved corrected review target The archival source above is not asserted equivalent to this replacement.

For a feasible q -acceptant 1-unstable choice rule on a positive-capacity universe larger than q , exact variability one holds if
 $\hookrightarrow$ and only if the rule is q -representative, equivalently characterized by one strict total order.

Archival source anchor: cited publication, lines 315-323

Recorded basis DGD source clarification: variability one is the one-total-order characterization on the nondegenerate queue domain.

Expanded PaperInterface specification

```
∀ {α : Type u_1} [inst : DecidableEq α] [inst_1 : Fintype α] {q : ℕ}
{C : DGD26AdmissionsPredictability.PaperChoiceRule α},
DGD26AdmissionsPredictability.paper_choice_function_feasible C  $\hookrightarrow$ 
DGD26AdmissionsPredictability.paper_definition_q_acceptance q C  $\hookrightarrow$ 
DGD26AdmissionsPredictability.paper_definition_d_instability 1 C  $\hookrightarrow$ 
0 < q  $\hookrightarrow$ 
q < Fintype.card α  $\hookrightarrow$ 
(DGD26AdmissionsPredictability.paper_definition_variability_exactly 1 C  $\leftrightarrow$ 
DGD26AdmissionsPredictability.paper_definition_q_representativeness q C)
```

Lean proof endpoint (built separately)

DGD26AdmissionsPredictability.PaperInterface.paper_variability_one_iff_single_order_statement

Recorded source-to-Spec screening Verdict: not recorded

Reason: not recorded

Reviewer check: ☐ Matches approved corrected target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation